

[深入浅出] 动态规划五讲

作者: [RESOT](#)

XCPC萌新互助进步群 [2](#):174495261

前言

本文适用于刚刚入门动态规划但是对动态规划理解不全面的同学，或是从来没学过动态规划的同学。很多同学都在动态规划这一章节卡了很长时间，例如不懂 DP 为什么要这么写，为什么这样写可以得到正确答案。

本文将尽可能地对 DP 进行深入浅出式讲解，我会用尽可能少的篇幅来为同学讲解我对 DP 的一些理解。

此外，因为本文章为入门向文章，所以内容都为大多数情况，并非包含了全部的情况。

我会分为 **DP 问题的分解**、**DP 的性质**、**如何设计 DP 模型**、**DP 问题的泛化**、**DP 方案的选取**来分为五讲进行讲解。值得一提的是，每个章节并非是十分独立的章节，为了保证由浅入深、循序渐进的学习顺序，我会在一些章节对前面的知识进行补充。

再次强调，DP是一个非常吃熟练度的算法，仅仅看课程学习是远远不足的，请务必通过大量刷题来提升自己对于DP的掌握。

第一讲 三个子问题与状态的转移

[P1192 台阶问题](#)

在这一讲中，我们将重点放在：将DP问题转换成三个子问题。深入了解状态之间的转移关系。

将 DP 问题转换成三个子问题

关于 DP 问题，我将其分成了三个子问题：**定义**、**初始化**、**转移**。其中最关键的是定义的设计。

1. 定义：我们来定义一个数组 $dp[i]$ 来表示我们走到第 i 个台阶时的方案数。
2. 初始化： $dp_i = 0$ 。特别的，当我们一个台阶都不走，一直处于 0 个台阶的状态也算一个方案，所以有 $dp_0 = 1$ 。
3. 转移：考虑第 i 个台阶可以分别从第 $i - 1$ 个台阶和第 $i - 2$ 个台阶直到第 $i - k$ 个台阶走到。所以走到第 i 个台阶时的方案总数应该是我们走到这些台阶的方案数的总和。总和的原因是因为我们可以从第 $i - k \leq j \leq i - 1$ 个台阶直接走到第 i 个台阶（加法原理）。综上所述，我们可以得出转移方程：
$$dp_i = \sum_{j=i-k}^{i-1} dp_j。$$

为什么这样定义？为什么这样初始化？为什么这样转移？

这三个问题我们将在第三讲时进行详细地介绍。对于第一讲，我们仅仅了解将DP分解成三个子问题后进行转移即可。

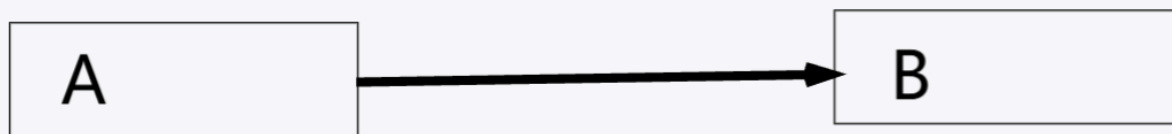
当我们解决了三个子问题后，我们将初始化和转移翻译成代码就能够解决这道DP问题了。此外，我不建议去有更多的考虑，仅仅思考解决这三个子问题就够了。

关于问题询问的走到第 N 个台阶的方案数，回顾我们的定义：**我们来定义一个数组 $dp[i]$ 来表示我们走到第 i 个台阶时的方案数**。那么答案其实就是 dp_n ，输出即可。

深入了解状态之间的转移关系

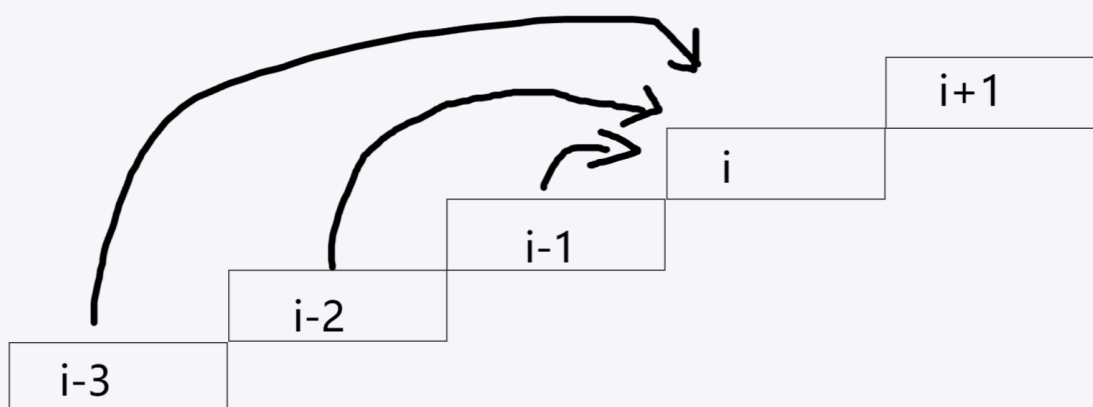
我个人认为想要了解 DP，去了解状态的抽象和转移是必不可少的关键步骤。

若状态 A 指向了状态 B ，那么说明计算状态 B 的 dp 值是要计算状态 A 的影响的。



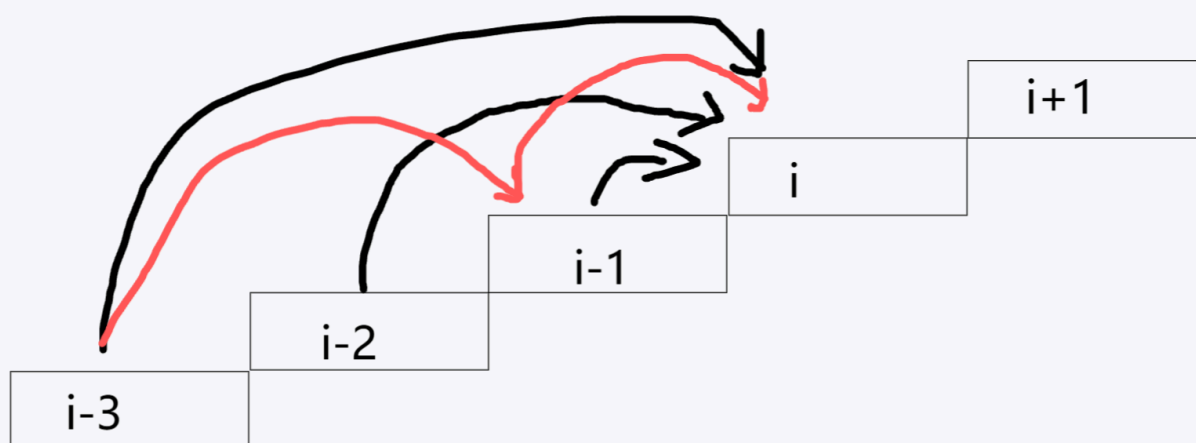
例如我们定义了 dp_i 为到达状态 i 的最大值，计算 dp_B 时，我们发现可以由状态 A 转移得到，那么有 $dp_B \leftarrow \max(dp_A, dp_B)$ 。

以本题为例，假设 $k = 3$ 的情况，考虑当前第 i 个台阶是从哪些台阶转移过来。



这里需要注意，当我们考虑第 i 个台阶的方案数时，第 $i + 1$ 个台阶及以后的台阶我们都不对其进行任何考虑。

同样的，也不需要去考虑 $i - 3$ 走到 $i - 1$ 再到 i 的方案是怎么统计进去的（即图中红色箭头）。



不需要考虑的原因很简单：红色箭头的方案已经被统计到了 dp_{i-1} 中。这里我建议大家回过头去看一眼我们这道题 dp 的定义。

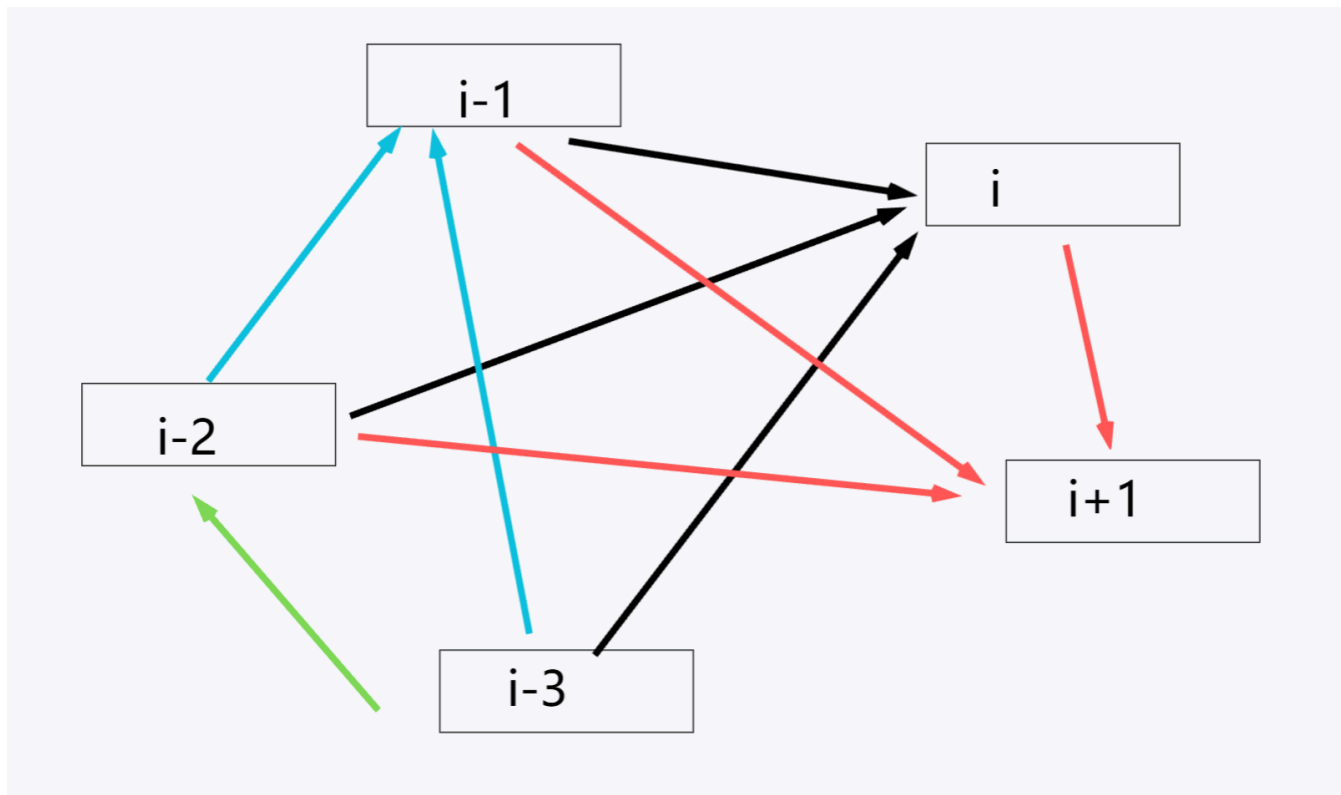
接下来，我们将台阶的位置重新分布，为了更直观的阐述**状态之间的转移**，我先将所有状态之间可转移的关系用不同颜色的箭头连接起来。

首先我们先定义**状态**：我们处于第 i 个台阶就称之为第 i 个状态。

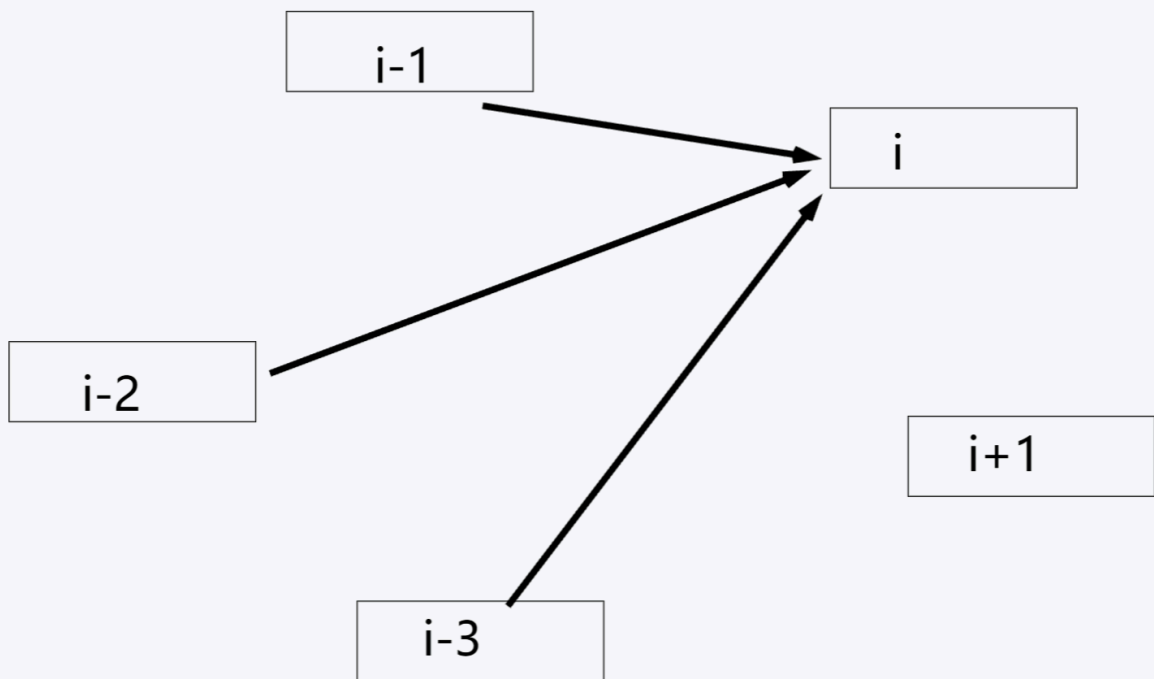
接下来考虑**状态之间的转移关系**。可以看到图中相同颜色的箭头用来表明有哪些状态可以转移到箭头所指向的状态。

例如蓝色箭头：第 $i - 1$ 个状态可以由第 $i - 2, i - 3, i - 4$ 三个状态转移得到，由于图中没画出第 $i - 4$ 个状态，所以只有两个蓝色箭头。

这样，我们就将台阶问题抽象成了**状态的关系图**。



接下来我们仅仅考虑状态 i 的计算，也就是仅保留黑色箭头。



这就帮助我们直观地理解 dp_i 是由哪些信息计算得来的。

再次强调，考虑转移时，我们仅仅考虑状态 A 能否直接到达状态 B ，若能，便考虑状态 A 对状态 B 的影响。

例如台阶问题，你不需要考虑第 $i - 2$ 个台阶是怎么到第 i 个台阶的，只要第 $i - 2$ 个台阶能一步到达第 i 个台阶，你就可以将状态 $i - 2$ 向状态 i 进行连线。

代码

```
1  int n, k;
2  int dp[100001]; // 定义 dp[i] 来表示我们走到第 i 个台阶时的方案数。
3
4  int main() {
5      cin >> n >> k;
6      dp[0] = 1; // 当我们一个台阶都不走，一直处于 0 个台阶的状态也算一个方案，所以有 dp[0] = 1。
7      for (int i = 1; i <= n; i++) { // 我们枚举 i 来计算我们当前要走到第 i 个台阶的方案数。
8          for (int j = i - k; j <= i - 1; j++) { // 枚举 j 来表示要从第 j 个台阶走到第 i 个台阶，那么方案数就要统计到 dp[j] 中。
9              if (j < 0) continue; // 我们不可能从第 -1 个台阶进行移动，所以当 j < 0 时应该直接跳过。
10             dp[i] = (dp[i] + dp[j]) % 100003; // 注意本题有取模操作。
11         }
12     }
13     cout << dp[n]; // 回顾我们的定义，可以发现答案就是 dp[n]。
14     return 0;
15 }
```

第二讲 DP的两个性质

最优子结构和无后效性

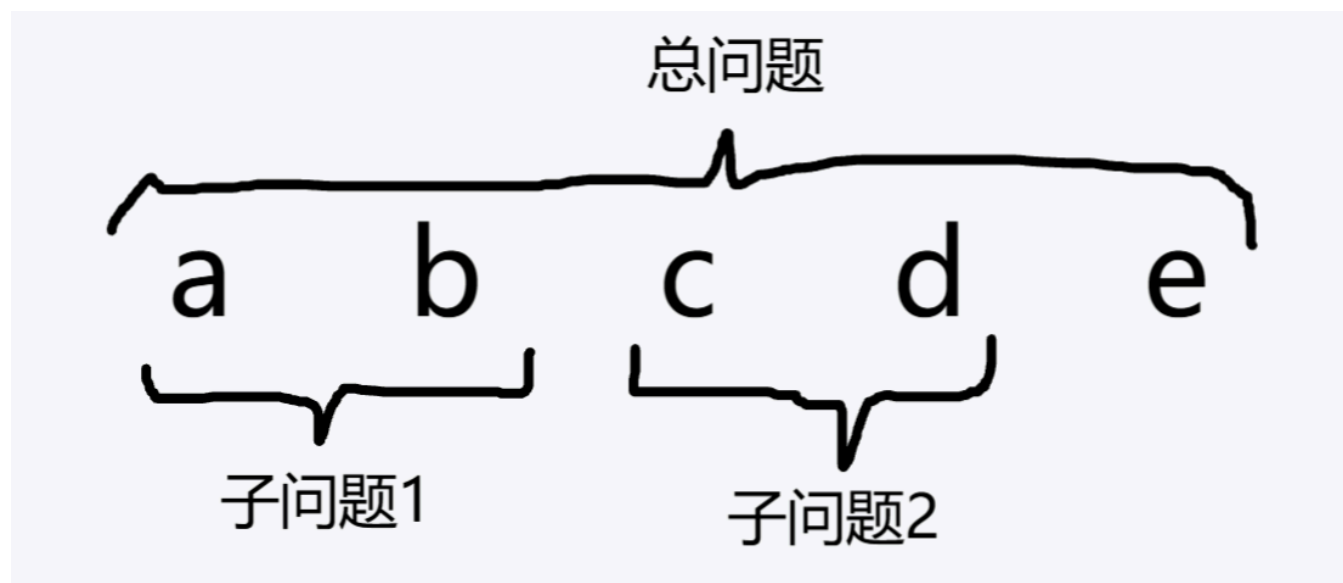
实际上，部分题目其实没有难在DP该如何设计，而是难在**如何注意到这道题是DP问题**。

一个很讨巧的方法：对于任何一道题目，我们都先把他当作DP问题来尝试性地去解决一下。

我们想要分析出这道题可以通过DP来解决，在这之前我们就一定要理解DP的性质——**最优子结构和无后效性**。

最优子结构：总问题的最优答案可以由子问题的最优答案计算得到。

举个例子：假设我有五个值，我想知道全部五个值的最大值，那么我可以先求出前四个值的最大值，在和第五个进行比较。我也可以求出前两个的最大值，第三个第四个值的最大值，这两个最大值和第五个进行比较的最大值就是我们的答案。其中求前两个的最大值、求第三个第四个值的最大值分别是两个子问题的最优答案。



最优子结构告诉我们，一个问题能否使用动态规划，关键在于它能否被拆分为若干子问题，并且总问题的最优解能够由这些子问题的最优解得到。至于状态应当按照什么顺序计算，则还需要进一步分析状态之间的依赖关系，保证当前状态能够由已经求出的状态推出。

无后效性：一旦状态确定，后续决策只与当前状态有关，而与到达这个状态的过程无关。

举个例子：同样是刚刚求最大值的例子。若规定选择图中的 a 就不能选择 d ，那么在后续判断是否可以选 d 时，就不能只看当前状态，还必须知道之前是否选择过 a 。这说明后续决策依赖于历史信息，也就是出现了后效性。

最优子结构和无后效性可以帮助我们分析出这道题可以通过动态规划求解。同样的，对于我们设计出的DP的定义、初始化和转移，那么也一定要满足**最优子结构和无后效性**。

对于相同的题目采用不同的dp定义，两个性质也许会不一样。

例如一个经典问题：输入 n, k 。表示给你 n 个数字，让你选任意个数字，选出的数字的和能恰好为 k 。

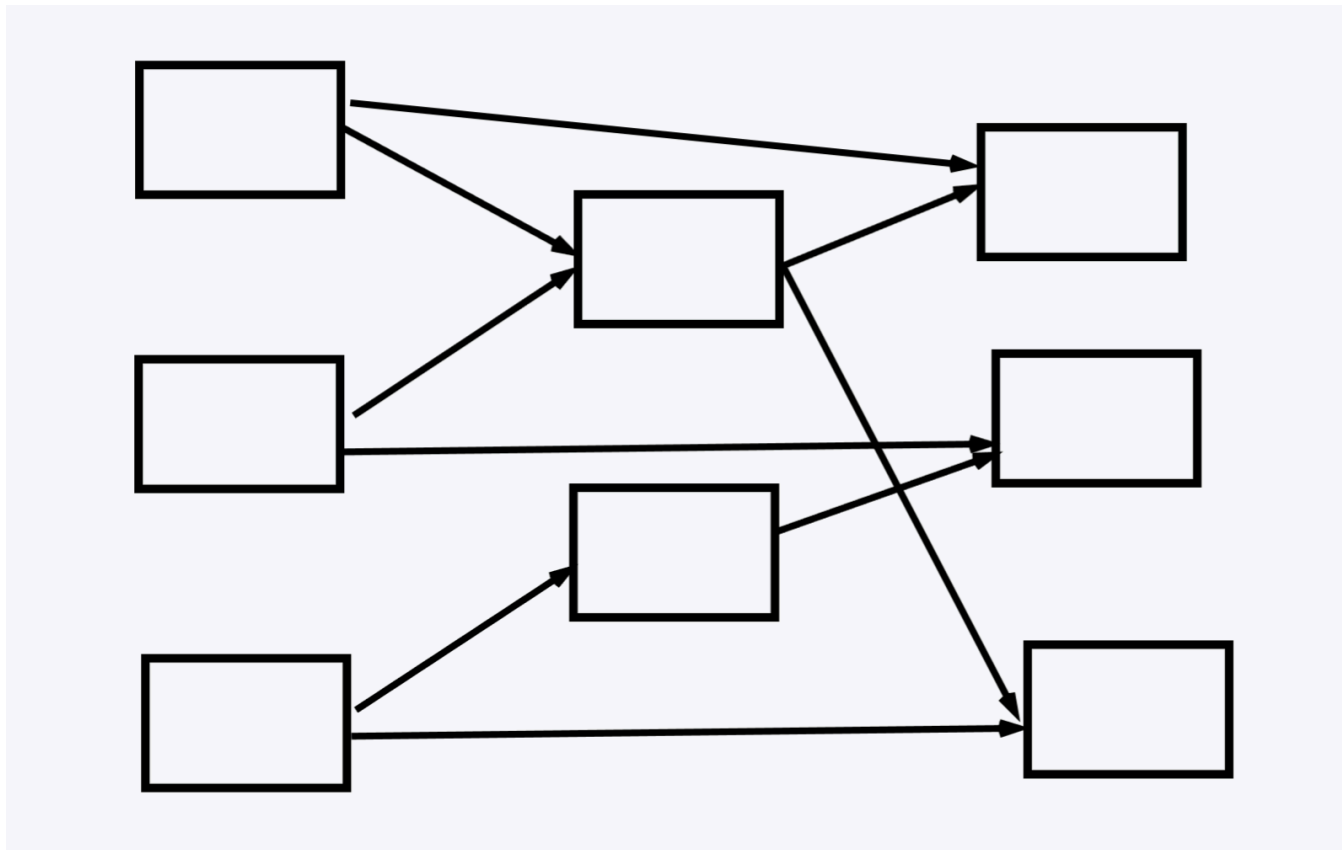
如果你定义 $dp_{i,0/1}$ 为前 i 个元素中，第 i 个元素不选/选可以得到的和，那么做这道题会非常麻烦。因为你无法确定不选/选第 i 个数是否对全局是最优的。这就是不满足最优子结构。

但是如果你的dp定义为： $dp_{i,j}$ 前 i 个数中，选择若干个数的和是否可以组合为 j ，那么你只需要判断 $dp_{n,k}$ 是否为 True 即可。因为你将前 i 个数可以组合出的所有和都表示了出来，所以是满足最优子结构的。

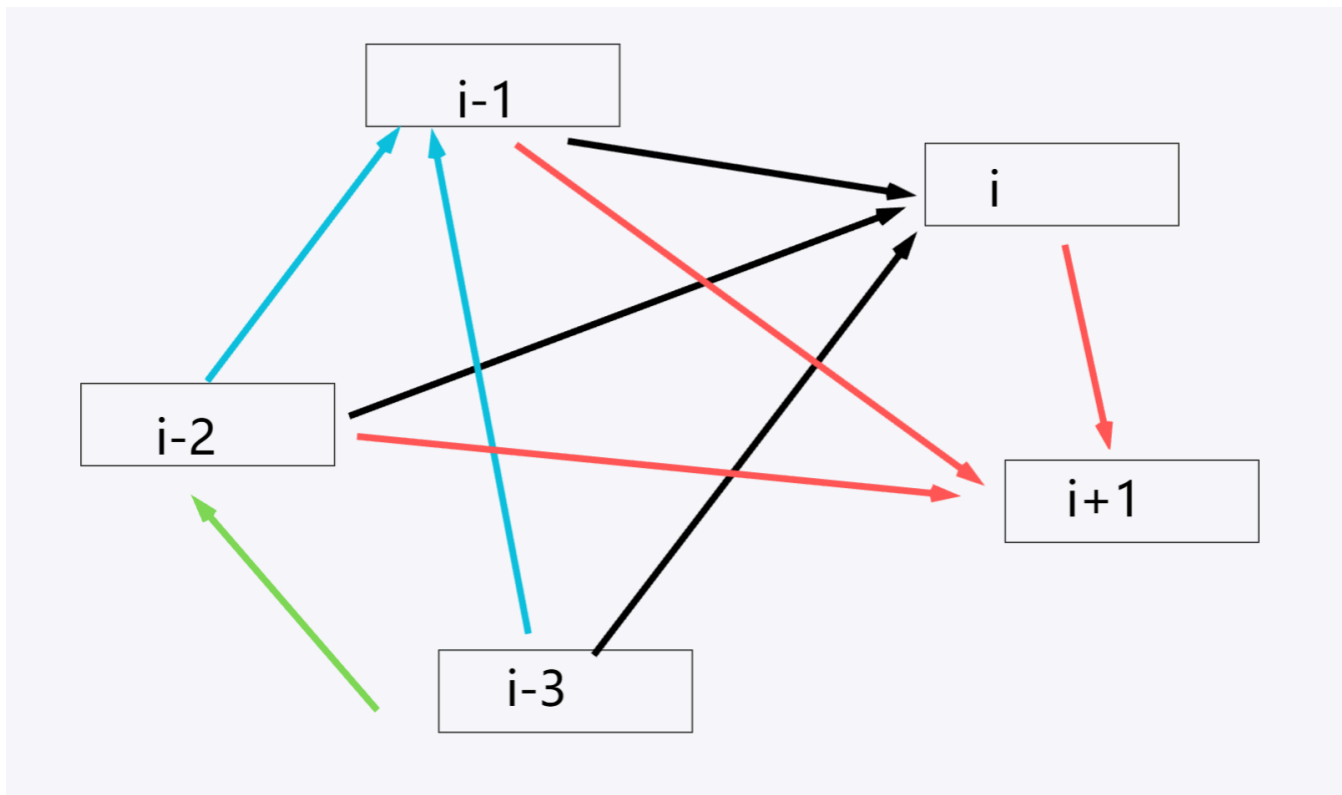
所以如果你当前考虑的dp模型并不满足最优子结构和无后效性，可以考虑去尝试其他的模型，或许就满足了。

从状态的角度理解

我们进行转移时，应该保证状态之间的转移是**无环**的（Dag 图），也就是设计出来的状态依赖关系，必须能按某种顺序被计算出来，例如：



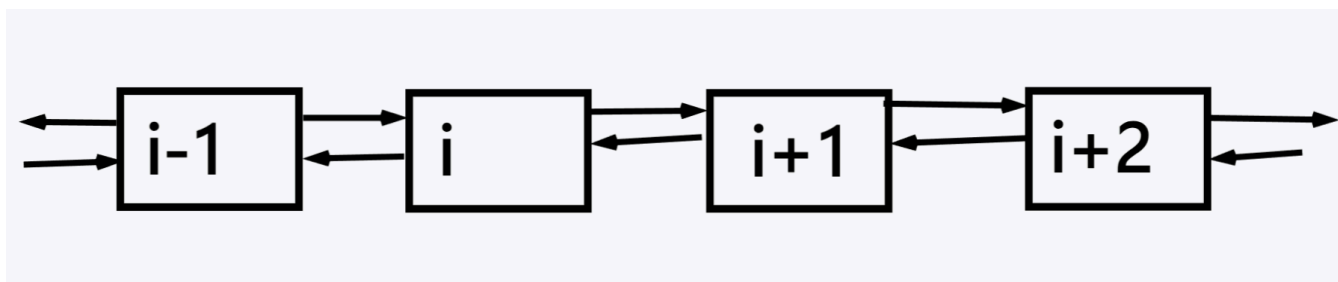
例如第一讲中的例题状态：



可以发现我们想要进行有效的状态转移需要保证状态之间的转移是无环的。

例如，我们定义 dp_i 为前 i 个元素通过操作得到的最优答案，但是我们的转移方程为 $dp_i = dp_{i-1} + dp_{i+1}$ 。

此时状态转移图为：



我们要计算 dp_i 的值就必须先知道 dp_{i-1}, dp_{i+1} 的值，但是我们想要知道 dp_{i-1} 的值就必须先知道 dp_i 的值，陷入了无限嵌套的循环。

所以我们在设计 DP 模型时应该保证状态之间的转移是无环的。

第三讲 如何设计 DP 模型

如何设计

一个讨巧的方法：你可以将自己做题时见过的所有模型都记住，做题时可以将你记住的模型挨个套进去试一遍，如果都不合适，那么你可以考虑放弃该题了。

关于定义：有多少个需要知道的信息就开多少维数组，数组的值一般是题目询问的答案。个人认为 DP 问题很大一部分难点就在于如何设计出符合题目需求的 DP 定义。

常见的几种定义：（仅仅是作为举例，具体题目务必具体分析）

1. dp_i 前 i 个元素可以得到的最大值/最小值/方案数。
2. $dp_{i,0/1}$ 前 i 个元素中，第 i 个元素不选/选的方案可以得到的最大值/最小值/方案数。
3. $dp_{i,j,k}$ 前 i 个元素使用了 j 次操作1使用了 k 次操作2 可以得到的最大值/最小值/方案数。
4. $dp_{i,j}$ 前 i 个元素选 j 个可以得到的最大值/最小值/方案数。
5. $dp_{i,j}$ 前 i 个元素是否可以组合出 j 。
6. $dp_{i,j}$ 前 i 个元素剩余空间为 j 可以得到的最大值/最小值/方案数。（背包dp）
7. dp_u 以 u 为根的子树中可以得到的最大值/最小值/方案数。（树形dp）
8. $dp_{l,r}$ 区间 $[l, r]$ 可以得到的最大值/最小值/方案数。（区间dp）
9. dp_s 选择的集合为 s 可以得到的最大值/最小值/方案数。（状压dp）
10. dp_u 以 u 为根时可以得到的最大值/最小值/方案数。（换根dp）

值得注意的是，在设计定义时，你应该特别留意题目的数据范围和是否满足最优子结构和无后效性。在设计定义时切记依据题目分析出的性质来设计模型，而非生搬硬套。

关于初始化：和题目询问反着来就可以了。比如问某某操作得到的最大值，那么你就设置成无穷小；问最小值，你就设置成无穷大；问方案数，你就全部设置成 0。接下来，你自己手算出最初的状态/最开始的一两步，定义的 DP 数组的值即可。

依照特定的题目，有可能不能设置成无穷大/无穷小，做题还需要具体问题具体分析，这里仅作入门所以会尽可能地减少思维难度来进行讲解。

关于转移：首先你要将问题的状态全部抽象出来，将各个状态之间的联系用箭头连起来，然后去考虑从状态 A 转移到状态 B 时会对 dp 值有什么影响。为了一般性，我们还应该从中间的一步骤进行考虑，而非从第一步或最后一步。

状态的转移也应该具体问题具体分析，例如：

1. 若转移是求和运算： $dp_B \leftarrow dp_B + dp_A$ 。
2. 若转移是求最大值： $dp_B \leftarrow \max(dp_B, dp_A)$ 。
3. 若转移是求最小值： $dp_B \leftarrow \min(dp_B, dp_A)$ 。
4. 若转移是求状态 A 到状态 B 加上 B 点的贡献的最大值： $dp_B \leftarrow \max(dp_B, dp_A + a_B)$ 。其中 a_B 是 B 点的贡献。

在这里，我想要再次强调一下把状态从问题中抽象出来。顺序是我们先想个符合无后效性和最优子结构的DP模型，然后依照DP的定义将题目中每个点的每个情况都列举出来作为状态，然后去考虑每个状态之间是否可以依照题目中提供的操作进行转移，这一点在下面的例题我会继续说明。

关于答案：我们要回归定义。例如我们对 dp 数组的定义为 dp_i 表示以 i 为区间右端点的答案，若题目问的是以任意区间可以得到的最有答案。那么答案并非是 dp_n ，因为依照定义 dp_n 表示区间 $[l, n]$ 的答案，其中 $l \leq n$ ，这个区间是严格以 n 为右端点。但事实上最有的区间并非一定是以 n 为右端点。所以你需要单独用循环枚举区间的右端点来获得最优答案。例如下面的代码是以求最大值为例。

```
1 int ans = 0;
2 for (int r = 1; r <= n; r++) ans = max(ans, dp[r]);
```

例题

[P1541 \[NOIP 2010 提高组\] 乌龟棋](#)

考虑如何设计定义：

题目问的是最大得分，那么我们就令 dp 的值为最大值。接下来考虑维数和含义，那么就要看有多少个需要知道的信息：

1. 因为每到达一个点就要加上这个点的贡献，所以我们要知道当前在哪个点，所以加一维用于表示当前所在位置。 dp_i 表示当前在 i 点的最大值。
2. 由于卡牌数量有限，如果不知道当前用了哪些卡牌，就会出现使用超过限制数量的卡牌。所以对于每一类型的卡牌，都要加一维用于表示我们当前使用了多少张。 $dp_{i,one,two,three,four}$ 表示当前在 i 点使用了 one 张 1 号卡牌、使用了 two 张 2 号卡牌、使用了 $three$ 张 3 号卡牌、使用了 $four$ 张 4 号卡牌可以得到的最大值。

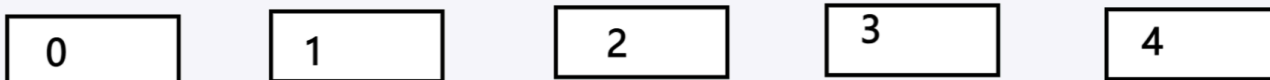
考虑设计初始化：

题目问最大值就把所有状态设为无穷小。由于最开始使用了 0 张卡牌处于 0 号位置，此时的得分为 0，所以有 $dp_{0,0,0,0,0} = 0$ 。

考虑设计转移：

首先将状态抽象出来，我们在这道题中有 $n + 1$ 个点，于是至少有 $n + 1$ 个状态表示我们在哪个位置。

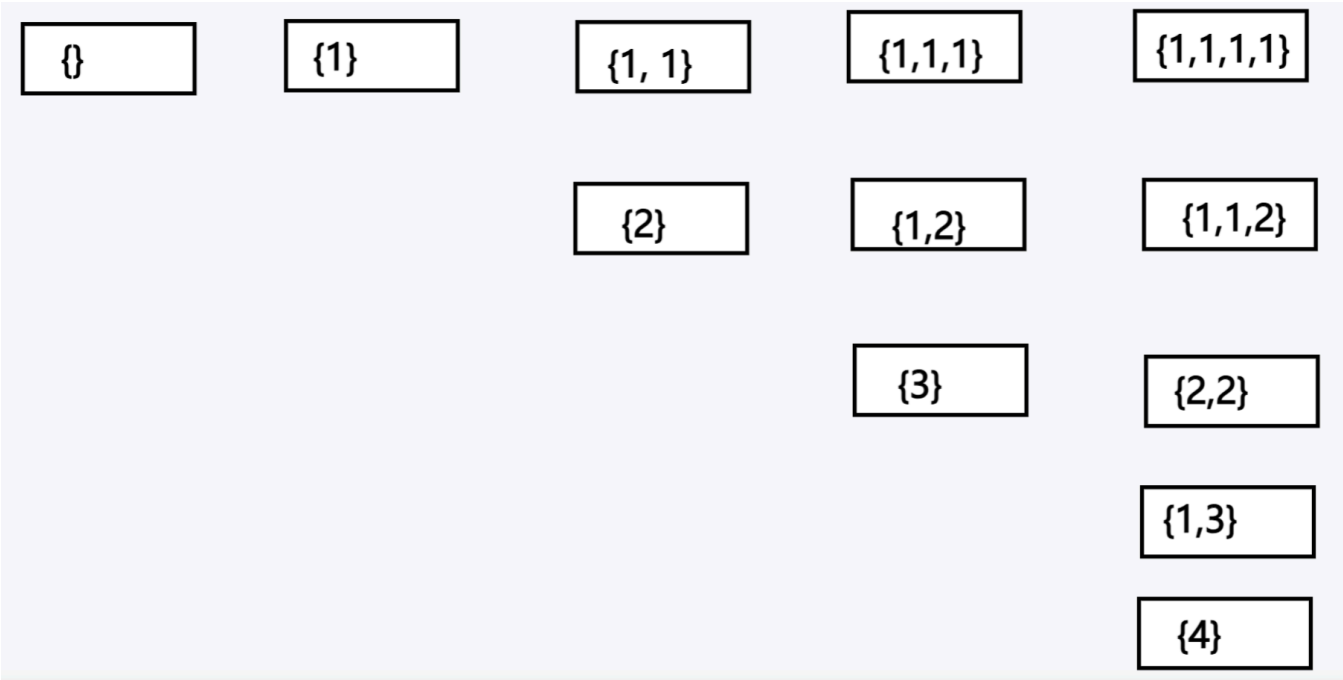
假设 $n = 4$ ，那么有 0, 1, 2, 3, 4 五个位置，即五种状态。



考虑转移，我们是通过使用卡牌的值来进行相应的移动，所以每个状态还应该包含到位置 i 使用了哪些卡牌。于是我们进一步将状态进行分解得到下图：

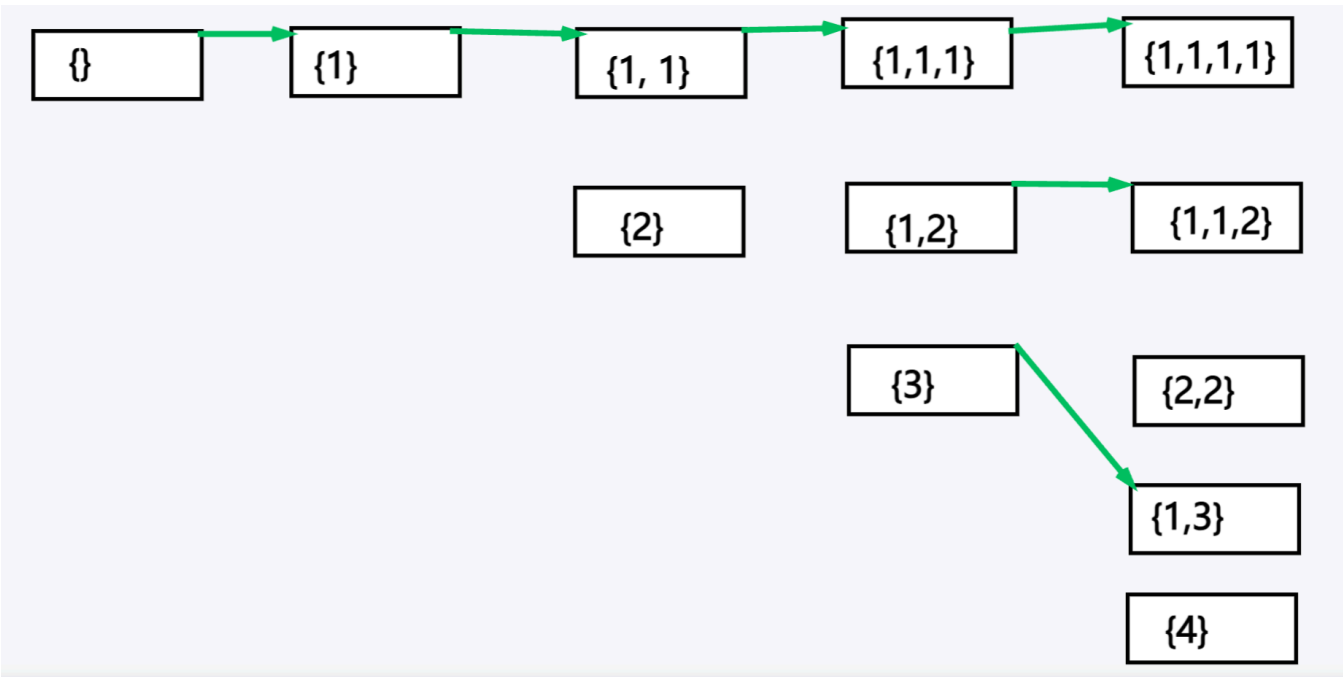
我们使用 $\{x, y\}$ 表示到达这个状态使用了 x, y 两张卡片。依据方框所处的列表示当前是在哪一个点，例如处于第 3 列的 $\{1, 1\}, \{2\}$ 两个状态分别表示使用了两张 1 号牌和使用了一张 2 号牌到达了位置点 2。

这样我们就实现了将状态从问题中抽象出来。

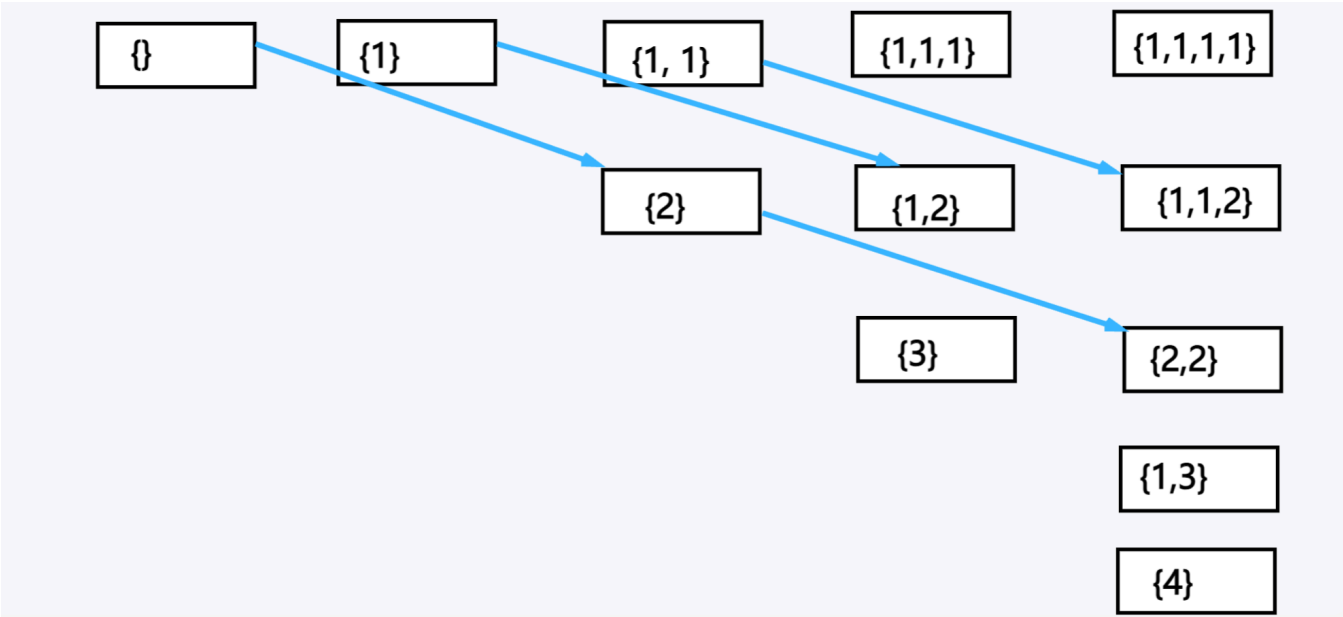


接下来考虑状态之间的转移。

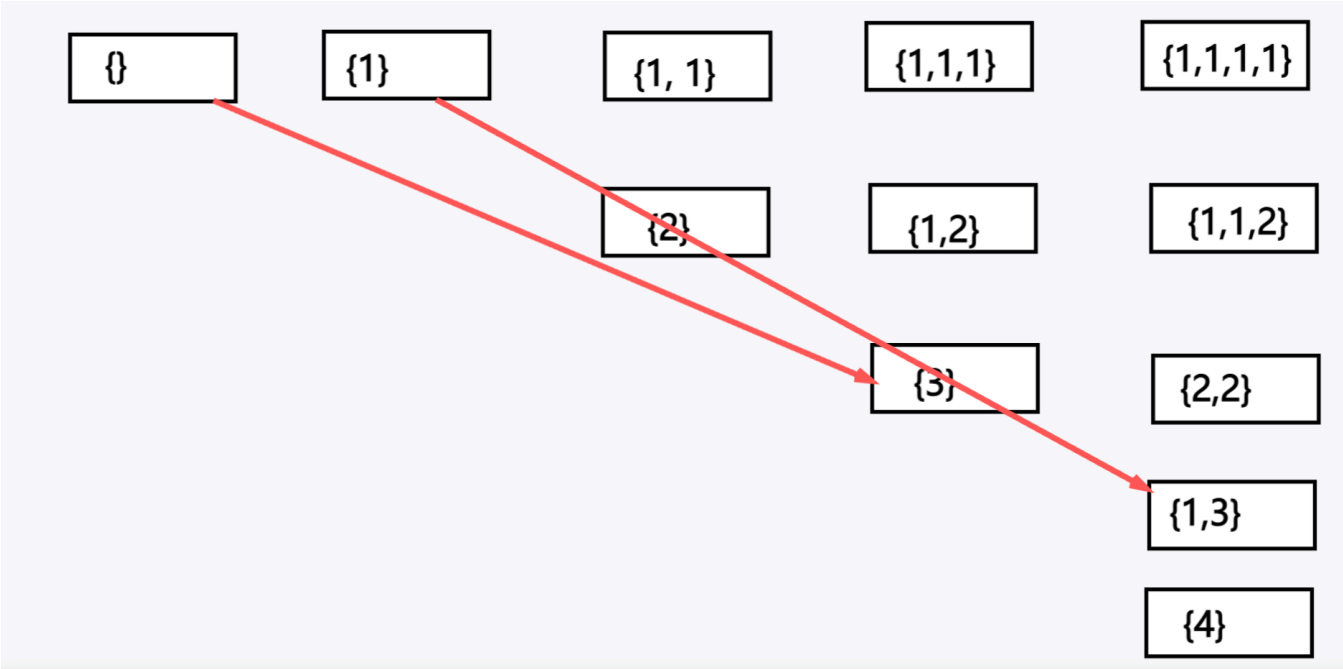
如果使用卡牌 1 会像下图的绿色箭头一样转移。



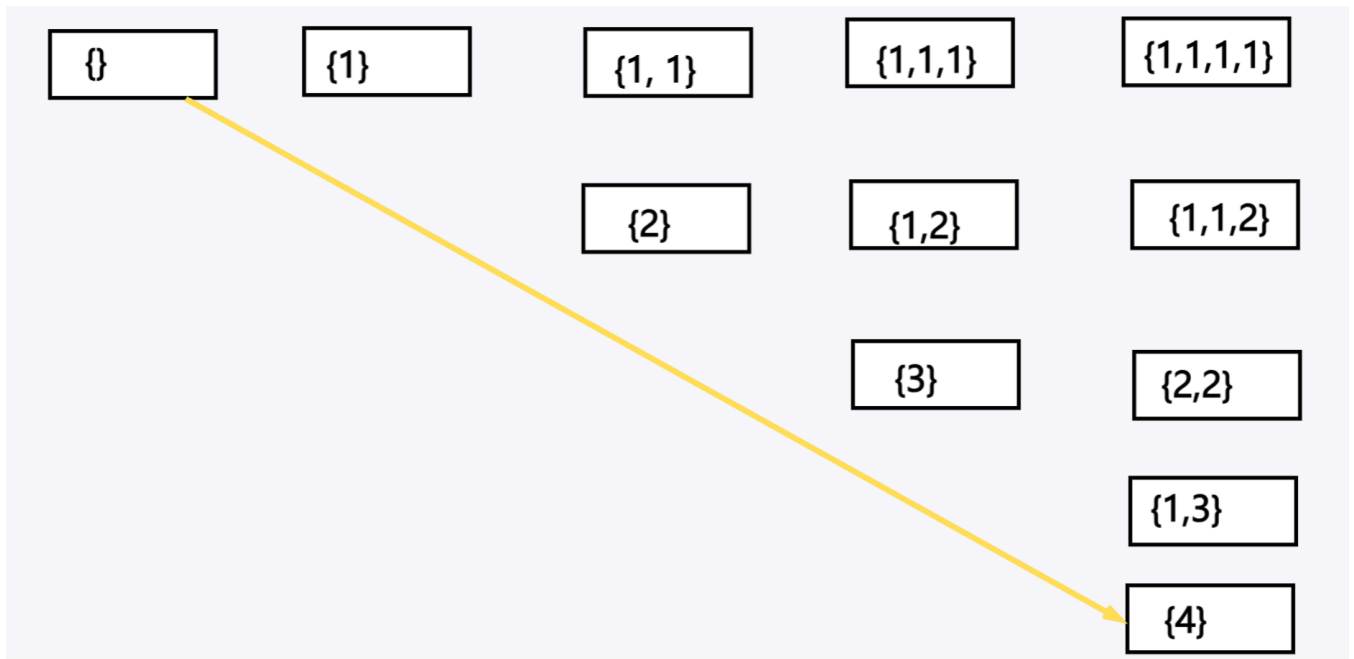
如果使用卡牌 2 会像下图的蓝色箭头一样转移。



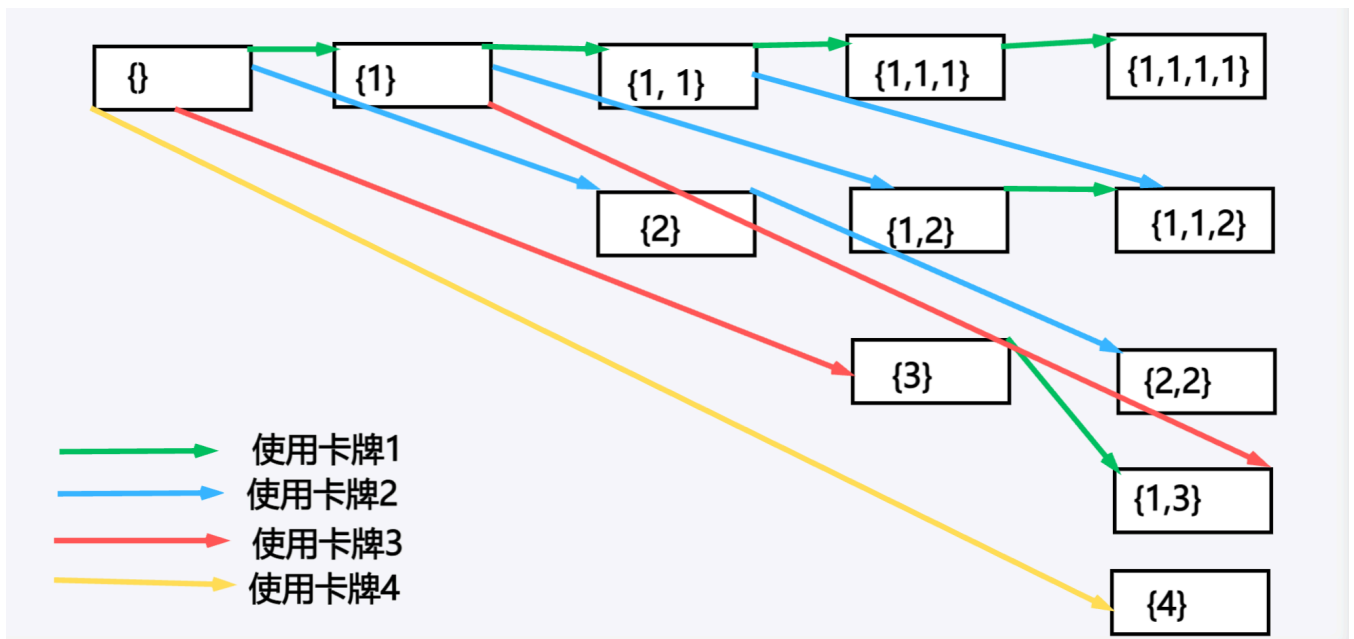
如果使用卡牌 3 会像下图的红色箭头一样转移。



如果使用卡牌 4 会像下图的黄色箭头一样转移。



将所有转移和状态全部呈现便是如下图所示：



现在假设我们在中间的某个步骤，设当前在 i 点使用了 one 张 1 号卡牌、使用了 two 张 2 号卡牌、使用了 $three$ 张 3 号卡牌、使用了 $four$ 张 4 号卡牌可以得到的最大值。

考虑当前状态是由哪些状态转移得到：

1. 使用了一张卡牌 1 转移的来：当前位置在点 i ，那么转移前的位置就在点 $i - 1$ ，使用的卡牌数量分别是 $one - 1, two, three, four$ ，该状态的值便为 $dp_{i-1,one-1,two,three,four}$ 。
2. 使用了一张卡牌 2 转移的来：当前位置在点 i ，那么转移前的位置就在点 $i - 2$ ，使用的卡牌数量分别是 $one, two - 1, three, four$ ，该状态的值便为 $dp_{i-2,one,two-1,three,four}$ 。
3. 使用了一张卡牌 3 转移的来：当前位置在点 i ，那么转移前的位置就在点 $i - 3$ ，使用的卡牌数量分别是 $one, two, three - 1, four$ ，该状态的值便为 $dp_{i-3,one,two,three-1,four}$ 。
4. 使用了一张卡牌 4 转移的来：当前位置在点 i ，那么转移前的位置就在点 $i - 4$ ，使用的卡牌数量分别是 $one, two, three, four - 1$ ，该状态的值便为 $dp_{i-4,one,two,three,four-1}$ 。

由于有最多四种可能性，依照 dp 设计的定义我们要求的是最大值，所以一定是从上述四种状态取最大的值来进行转移，且转移后要加上到达 i 点的贡献（设贡献为 a_i ），所以转移方程为：

$$dp_{i,one,two,three,four} = a_i + \max\{dp_{i-1,one-1,two,three,four}, dp_{i-2,one,two-1,three,four}, dp_{i-3,one,two,three-1,four}, dp_{i-4,one,two,three,four-1}\}.$$

我们解决了定义、初始化、转移，那么将公式翻译成代码就可以了。

但是我们现在的定义是无法通过这道题的，这是因为我们开了五维数组，空间已经高达 $350 \times 40^4 \approx 9 \times 10^8$ 。远远超出了题目的空间限制。

所以我们要进行优化，事实上我们并不需要单独开一维空间表示我们当前所在点的位置，因为当我们确定了每张卡牌使用了多少张后，可以通过公式 $one \times 1 + two \times 2 + three \times 3 + four \times 4$ 来算出我们当前所在位置，也就是说位置已经不是未知信息了，就不需要单独开一维空间表示。

所以我们的 dp 定义改为 $dp_{one,two,three,four}$ 表示当前使用了 one 张 1 号卡牌、使用了 two 张 2 号卡牌、使用了 $three$ 张 3 号卡牌、使用了 $four$ 张 4 号卡牌可以得到的最大值。隐含了我们当前所在的位置的信息。此时空间为 $40^4 \approx 2 \times 10^6$ ，空间复杂度符合要求。

最后考虑答案的统计，由于保证我们使用完所有的卡牌后会恰好到达终点，依据定义，答案就是 $dp_{one,two,three,four}$ 。

例题代码

补充：题目中实际上是以 1 号点作为起点，在上文的讲解中是以 0 号点作为起点，所以在实际代码中表示当前位置的计算应该额外 +1。

```
1  const int INF = 0x3f3f3f3f; // 用于表示无穷大
2  int n, m, a[1000001], cnt[10001]; // a[i] 表示到达 i 点会获得的分数，cnt[i] 表示值为 i 的卡牌
   的数量
3  int dp[50][50][50][50];
4  // 表示当前使用了 one 张 $1$ 号卡牌
5  // 使用了 $two$ 张 $2$ 号卡牌
6  // 使用了 $three$ 张 $3$ 号卡牌
7  // 使用了 $four$ 张 $4$ 号卡牌可以得到的最大值。
8  // 隐含了我们当前所在的位置的信息。
9
10 int main() {
11     cin >> n >> m;
12     for (int i = 1; i <= n; i++) cin >> a[i];
13     for (int i = 1; i <= m; i++) {
14         int x;
15         cin >> x;
16         cnt[x]++; // 使用 cnt[x] 表示值为 x 的卡牌有 cnt[x] 张
17     }
18
19     for (int one = 0; one <= cnt[1]; one++) // 枚举使用 one 张值为 1 的卡牌
20         for (int two = 0; two <= cnt[2]; two++) // 枚举使用 two 张值为 2 的卡牌
21             for (int three = 0; three <= cnt[3]; three++) // 枚举使用 three 张值为 3 的卡
   牌
```

```

22         for (int four = 0; four <= cnt[4]; four++) // 枚举使用 four 张值为 4 的
    卡牌
23         dp[one][two][three][four] = -INF; // 负的无穷大便是无穷小（不同于高数中趋
    近于 0 的无穷小）
24         dp[0][0][0][0] = 0; // dp 的初始化操作
25
26         for (int one = 0; one <= cnt[1]; one++) // 枚举使用 one 张值为 1 的卡牌
27             for (int two = 0; two <= cnt[2]; two++) // 枚举使用 two 张值为 2 的卡牌
28                 for (int three = 0; three <= cnt[3]; three++) // 枚举使用 three 张值为 3 的卡
    牌
29                     for (int four = 0; four <= cnt[4]; four++) { // 枚举使用 four 张值为 4 的
    卡牌
30                         int i = 1 + one * 1 + two * 2 + three * 3 + four * 4; // 用 i 表示当
    前所在位置
31                         int &now = dp[one][two][three][four]; // now 替代 dp[one][two]
    [three][four]（仅用作简写代码）
32                         if (one) now = max(now, dp[one - 1][two][three][four]);
33                         if (two) now = max(now, dp[one][two - 1][three][four]);
34                         if (three) now = max(now, dp[one][two][three - 1][four]);
35                         if (four) now = max(now, dp[one][two][three][four - 1]);
36                         now += a[i]; // 要加上到达 i 点后会获得的 a[i] 的贡献
37                     }
38
39         cout << dp[cnt[1]][cnt[2]][cnt[3]][cnt[4]] << "\n"; // 依据定义确定答案
40         return 0;
41     }
42

```

第四讲 DP问题的泛化理解

前言

接下来我将从背包、区间、树形的例题入手，来让大家明白对于不同类型的dp，差异也仅仅是我们换个定义而已。换句话说，我希望各位对于算法可以保持一种泛化的思想。

对于题目，我们都应该做到具体问题具体分析，因为做题时先入为主的话很容易造成后续理解上的误差，有可能用DP分析了半天最后才发现这题DP根本做不了。

对于每道例题我将其分成了概述、分析、DP设计、代码四个部分。其中概述部分是对该类型的DP问题的讲解，分析部分是对第三讲的DP的设计的巩固与加强。

我建议可以先只看DP设计部分，然后将公式翻译成代码直接交题，用代码部分提供的代码来比较自己写的是否正确，然后再看分析部分来巩固自己分析题目的理解。

背包 DP

概述

我不希望大家把背包 DP 记成：只能用来处理有一个背包和多种物品，然后去算这多种物品选取的最有价值。而是应该去思考该 DP 实际上解决的是：在有限的空间内，对于每种操作的代价与价值，去通过分析然后抽象出状态进行转移求解。虽然两者看起来差不多，但是如果只有背包的印象的话就很容易过拟合，认为只能解决带有背包字样的题目。

事实上，我很少考虑到一道题是不是背包 DP，通过第三讲的 DP 设计分析方法会比较自然的想到对应的 DP 设计。

例如一维背包，其实就是加了一重限制，就是线性 DP 在 dp_i 表示前 i 个物品的最优答案，但是有一重限制需要我们去表示，所以就在 DP 的定义中增加一个维度表示当前状态是否满足限制，例如 $dp_{i,j}$ 表示前 i 个物品的选择使用了 j 的空间。

如果是二维背包，那么继续在后面增加一个维度表示第二个限制即可，例如 $dp_{i,j,k}$ 表示前 i 个物品的选择使用了 j 的空间满足了 k 的限制。

正如我在前言中说的那样，我并不希望大家把 DP 当成死板的东西，DP 是一种相当灵活的思想。

分析

[P1048 \[NOIP 2005 普及组\] 采药](#)

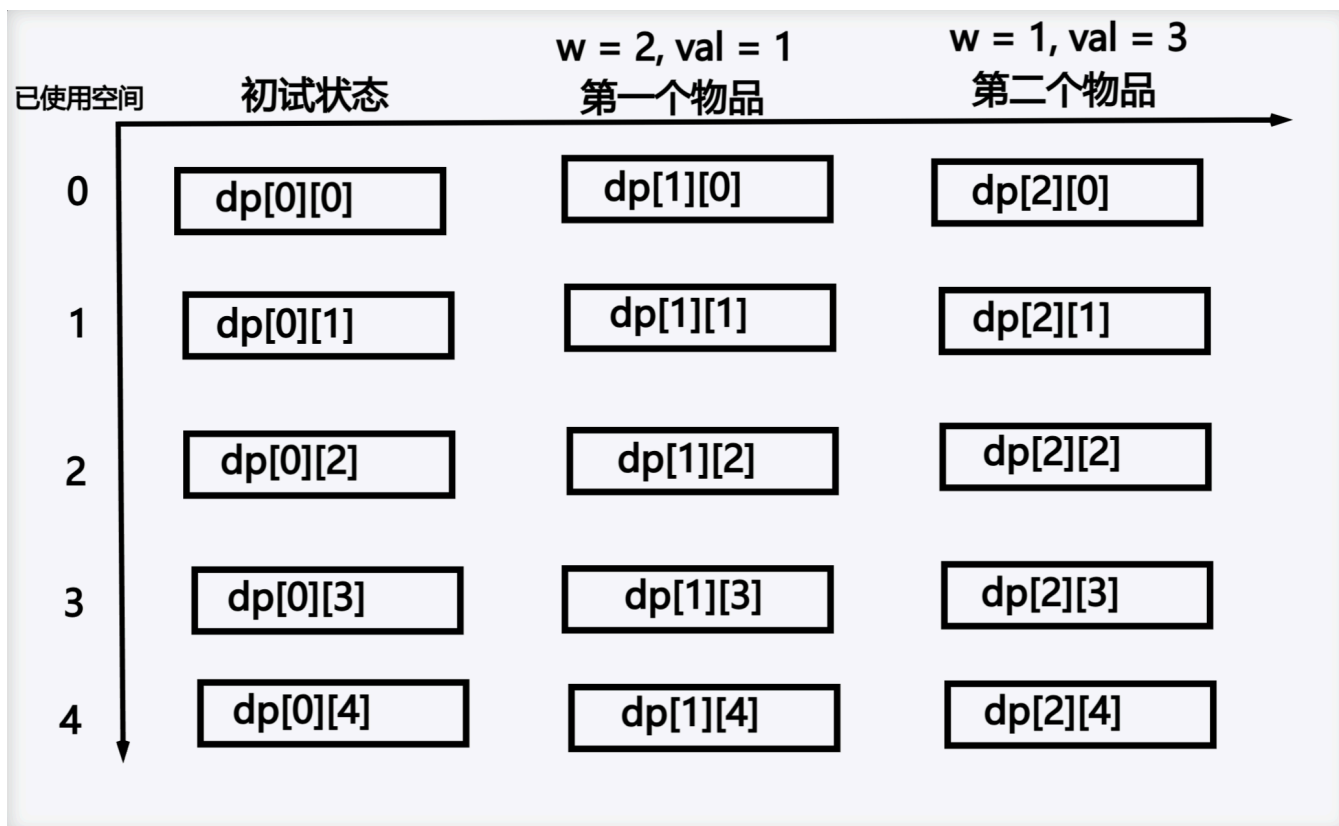
依照第三讲的内容进行分析。

问题问最大值，那么就设 dp 为选取物品后可以得到的最大值。

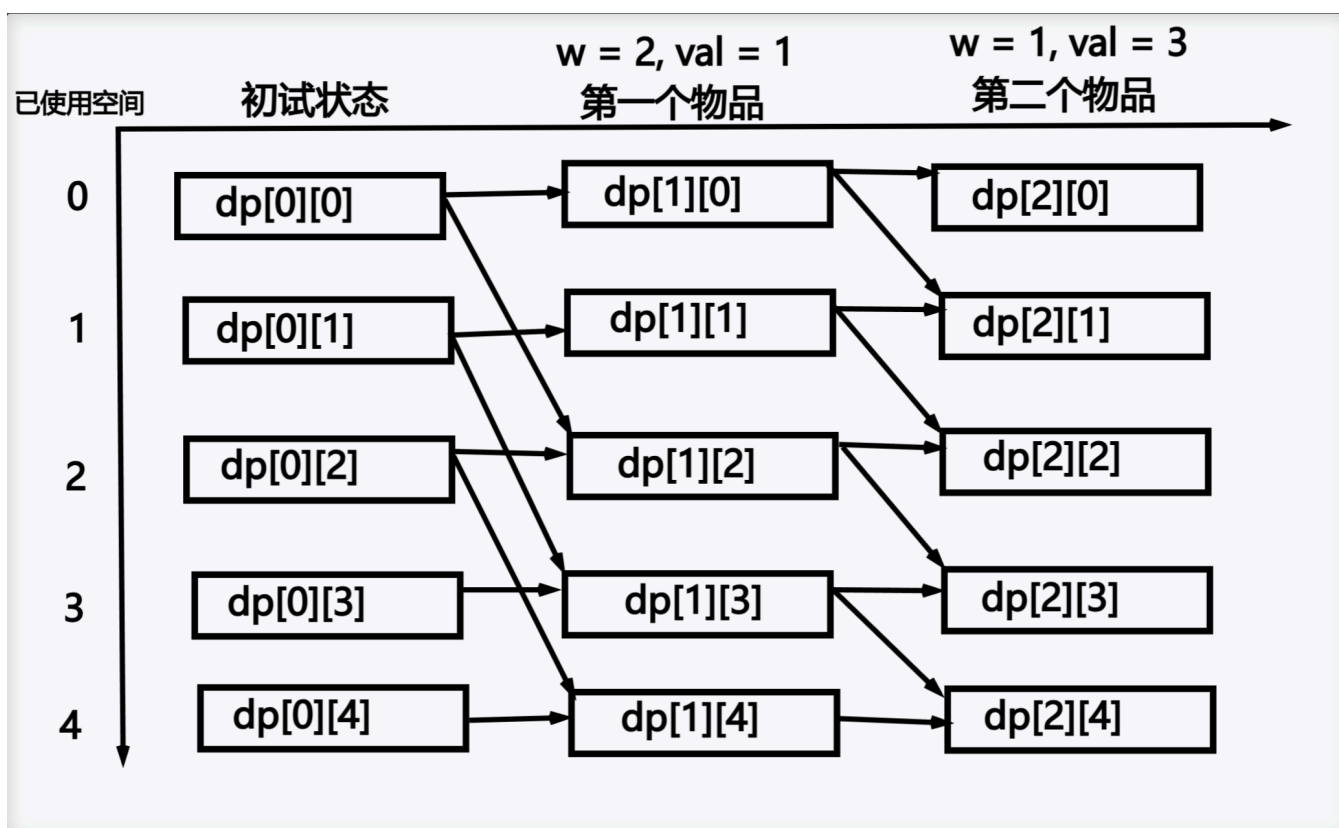
我们需要知道当前我们已经考虑了哪些物品的选取，所以增加一维。 dp_i 表示前 i 个物品的选取可以得到的最大值。

仅仅如此依旧不够，因为我们不知道选了几个物品后，背包能不能装得下，所以再加一维表示我们使用了背包的多少空间。 $dp_{i,j}$ 表示前 i 个物品的选取使用了 j 的空间可以获得的最大价值。

这样，题目中给出的两个限制条件： m 个物品和 t 的空间，我们通过增加 dp 的维度都表示出来了。再依据 dp 的定义来抽象出这道题的状态：



我们举例有两个物品的重量和价值分别为 $(2, 1)$, $(1, 3)$ ，可以得到如下转移图：



这里说明一下，并非每次做题都要将状态转移图画出来，实际上这里画图仅仅是为了方便讲授状态转移的概念，在自己做题时仅需考虑**当前状态可以转移到哪些状态**或者**哪些状态可以转移到当前状态**，保证状态之间的转移不会有环即可。

由于求最大值，所以我们先将所有状态设为负无穷。在最初的状态，我们一个物品都不选择，所以对于任意剩余空间，我们能得到的最大价值都是 0。

选择第 i 个物品会消耗 $weight_i$ 的空间来获得 val_i 的价值。设我们选取第 i 个物品后使用了 j 的空间，所以应该由前 $i - 1$ 个物品的选取方案中使用 $j - weight_i$ 空间的状态转移（预留出 $weight_i$ 的空间用来选择第 i 个物品）。所以有 $dp_{i,j} \leftarrow dp_{i-1,j-weight_i} + val_i$ 。此外，因为是由 $j - weight_i$ 转移得来，背包最少使用了 0 的空间，所以要保证 $j - weight_i \geq 0$ 。

如果不选择第 i 个物品，那么不需要考虑第 i 个物品对选择的影响，直接继承前 $i - 1$ 个物品的选择即可，所以有 $dp_{i,j} \leftarrow dp_{i-1,j}$ 。两者取其大，综合为 $dp_{i,j} \leftarrow \max(dp_{i-1,j}, dp_{i-1,j-weight_i} + val_i)$ 。

DP 设计

定义： $dp_{i,j}$ 表示前 i 个物品的选取使用了 j 的空间可以获得的**最大价值**。

初始化： $dp_{i,j} = -\infty, dp_{0,0} = 0$ 。

转移： $dp_{i,j} \leftarrow \max(dp_{i-1,j}, dp_{i-1,j-weight_i} + val_i)$ 。

代码

```
1  const int INF = 0x3f3f3f3f;
2
3  int t, m;
4  int val[1001], weight[1001];
5  int dp[101][1001];
6
7  int main() {
8      cin >> t >> m;
9      for (int i = 1; i <= m; i++)
10         cin >> weight[i] >> val[i];
11
12     // 初始化
13     for (int i = 0; i <= m; i++)
14         for (int j = 0; j <= t; j++)
15             dp[i][j] = -INF; // 首先将所有状态都设为负无穷
16     dp[0][0] = 0; // 最初一个物品都不选，价值为0
17
18     // 转移
19     for (int i = 1; i <= m; i++)
20         for (int j = 0; j <= t; j++) {
21             dp[i][j] = dp[i - 1][j]; // 不选第 i 个物品可以直接继承前 i - 1 个物品的选取
22             if (j < weight[i]) continue; // 选第 i 个物品要保证选择后背包的使用应该大于
weight[i]
23             dp[i][j] = max(dp[i - 1][j], dp[i - 1][j - weight[i]] + val[i]); // 两者取其
大
24         }
25
26     // 计算答案
27     // 由于 dp 的定义为 前 i 个物品使用了 j 空间的最大值，dp[m][t] 只能得到使用了恰好 t 空间的**最大
值
28     // 实际上有可能使用了 t - 1 空间才能得到最大值，所以要遍历一次使用了多少空间来统计最大值才可以
29     int ans = -INF;
30     for (int j = 0; j <= t; j++) ans = max(ans, dp[m][j]);
31     cout << ans << "\n";
32
33     return 0;
```


更进一步

事实上一道题也许会有多种解法，我们换个设计依旧可以解决这道题。

定义： $dp_{i,j}$ 表示前 i 个物品的选取使用了**不超过** j 的空间可以获得的最大价值。

初始化： $dp_{i,j} = -\infty, dp_{0,j} = 0$ 。

转移： $dp_{i,j} \leftarrow \max(dp_{i-1,j}, dp_{i-1,j-weight_i} + val_i)$ 。

由于我们定义变成了**不超过** j ，其表示 $[0, j]$ 的范围，所以答案的选取便是 $dp_m[t]$ 了。

事实上转移也可以写成 $dp_{i,j} \leftarrow \max(dp_{i-1,j}, dp_{i-1,j-weight_i} + val_i, dp_{i,j-1})$ ，也就是将 $dp_{i,j-1}$ 也考虑进去。

但是没必要，假设我们选了第一个商品，其重量为 2，在**恰好**的定义中，仅有 $dp_{1,2}$ 会被 $dp_{0,0}$ 转移得到更新，而对于 $dp_{1,3}, dp_{1,4}$ 等都不会更新，这是因为 $dp_{0,1}, dp_{0,2}$ 都为负无穷小（在初始化值为无穷大/无穷小时，我们可以理解为这个状态还未更新或是不存在这个状态），所以无法转移得到。在**不超过**的定义中，由于 $dp_{0,1}, dp_{0,2}$ 都改为初始值为 0，所以 $dp_{1,3}, dp_{1,4}$ 就可以更新。

或者可以从状态的角度理解，在**恰好**的定义中，转移图为：

第0个物品

第一个物品,

$dp[0][0]=0$

$dp[1][0]=-INF$

$dp[0][1]=-INF$

$dp[1][1]=-INF$

$dp[0][2]=-INF$

$dp[1][2]=1$

$dp[0][3]=-INF$

$dp[1][3]=-INF$

设第一个物品重量为2, 价值为1

其中蓝色箭头表示 $dp_{i,j} \leftarrow dp_{i-1,j}$ 的转移, 红色箭头表示 $dp_{i,j} \leftarrow dp_{i-1,j-weight_i} + val_i$ 的转移。

在**不超过**的定义中, 转移图就变成了如下:

第0个物品

第一个物品,

$dp[0][0]=0$

$dp[1][0]=0$

$dp[0][1]=0$

$dp[1][1]=0$

$dp[0][2]=0$

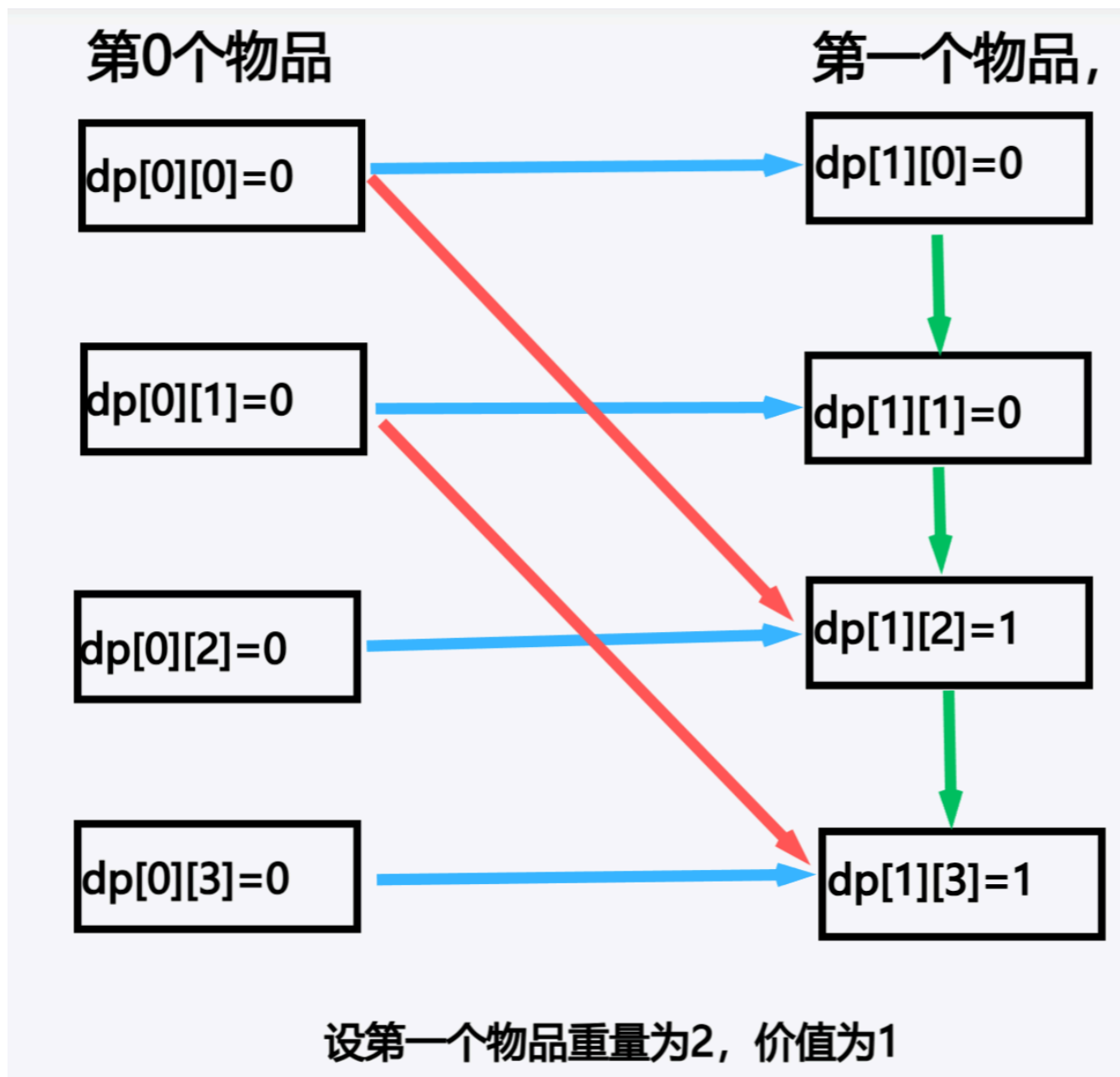
$dp[1][2]=1$

$dp[0][3]=0$

$dp[1][3]=1$

设第一个物品重量为2, 价值为1

当然也可以加一个 $dp_{i,j} \leftarrow \max(dp_{i,j}, dp_{i,j-1})$ 的转移:



区间 DP

概述

区间 DP 一个很显著的特点：一般允许 $O(n^3)$ 级别的时间复杂度。

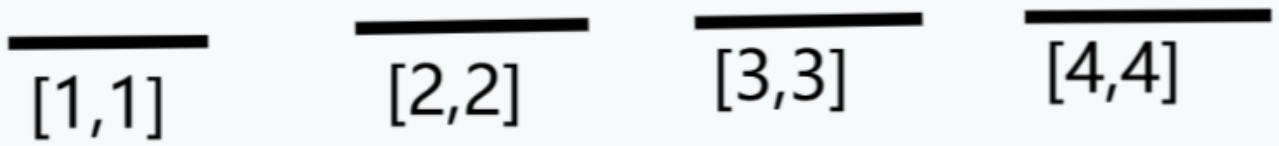
当然具体问题具体分析，有些题目会给出特殊操作，或者是合并具有特殊性，使得可以在 n^2 的时间复杂度完成转移。

实际上看到 n^3 的时间复杂度完全可以想一想这道题能不能用区间 DP 来解决，还有如果看到时间复杂度允许 2^n 的话就可以考虑一下能不能用状压 DP 来解决。

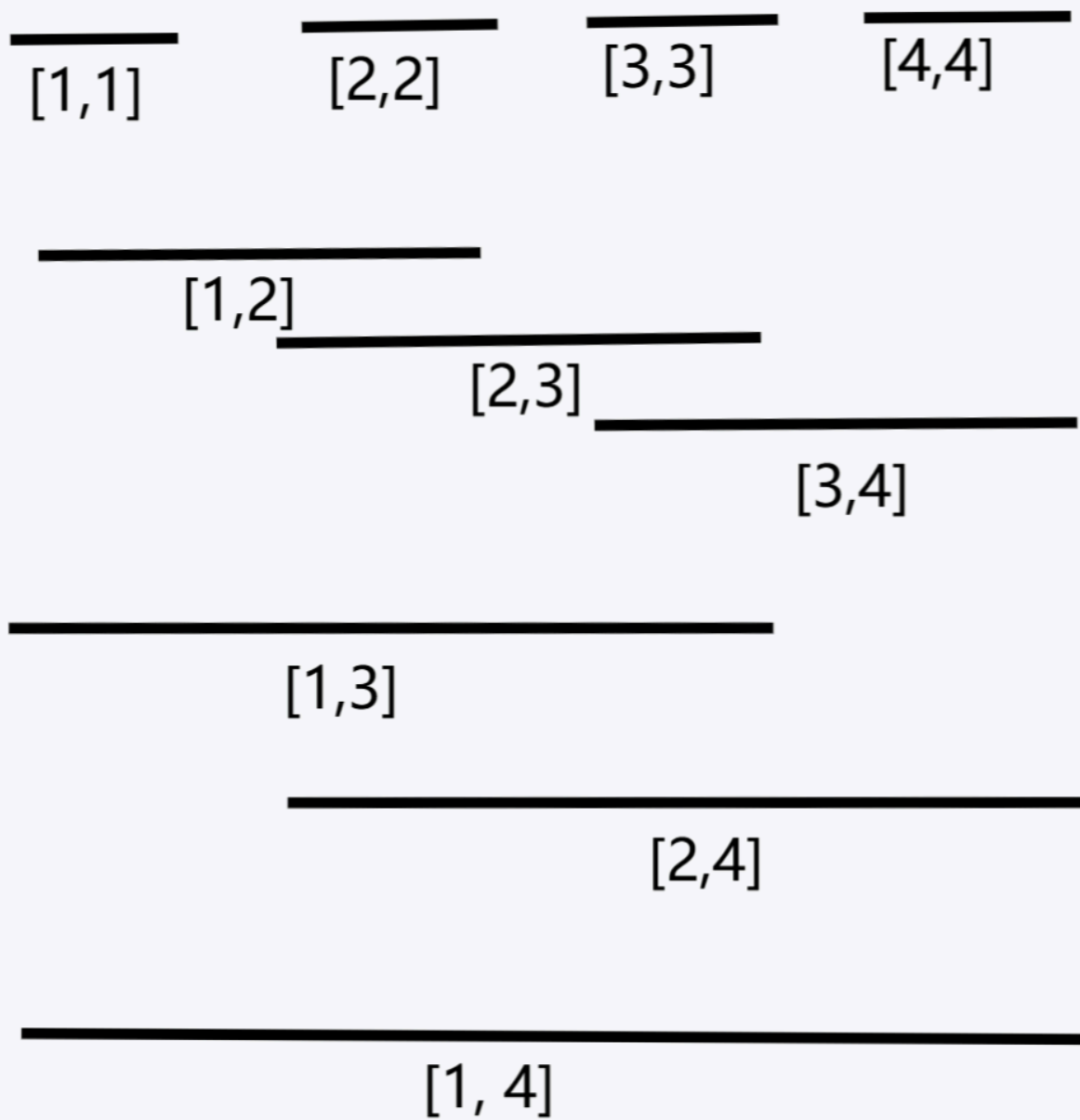
虽然这里我将区间 DP 拿出来单独讲，但是我依旧坚持对 DP 的理解应该保持泛化。区间 DP 的核心就是状态之间的转移并非从前到后、而是从范围小的区间转移到范围大的区间。

至于为什么区间 DP 的时间复杂度一般是 $O(n^3)$ 在下文会进行讲解，在这之前，我们先来看一下区间 DP 的状态转移，我们使用黑线来表示这个状态包含的元素集合，用黑线下方的中括号表示这个黑线包含了什么区间。

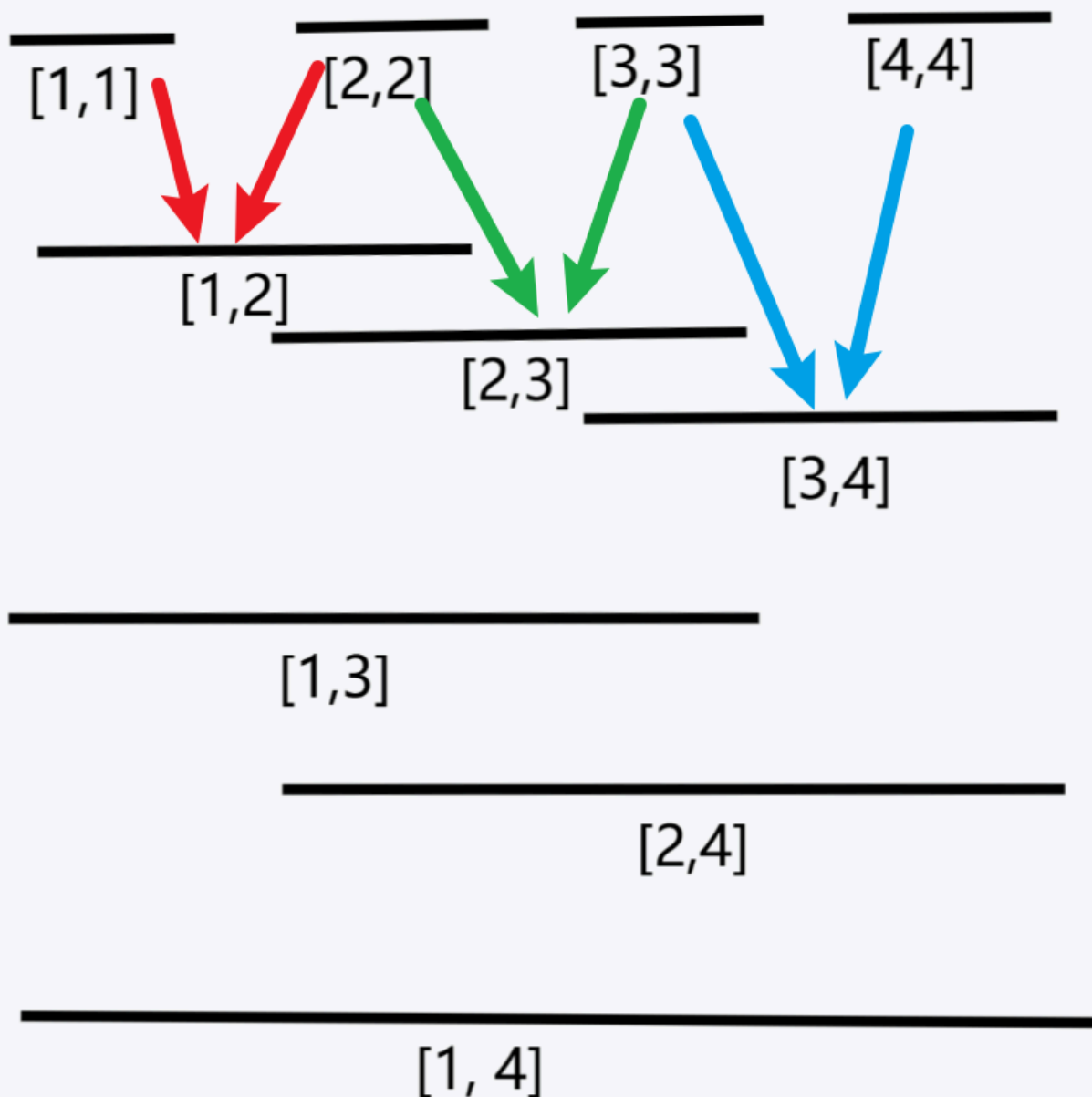
对于区间 DP 来说，最初始的状态是每个状态只有长度为一的元素自身：



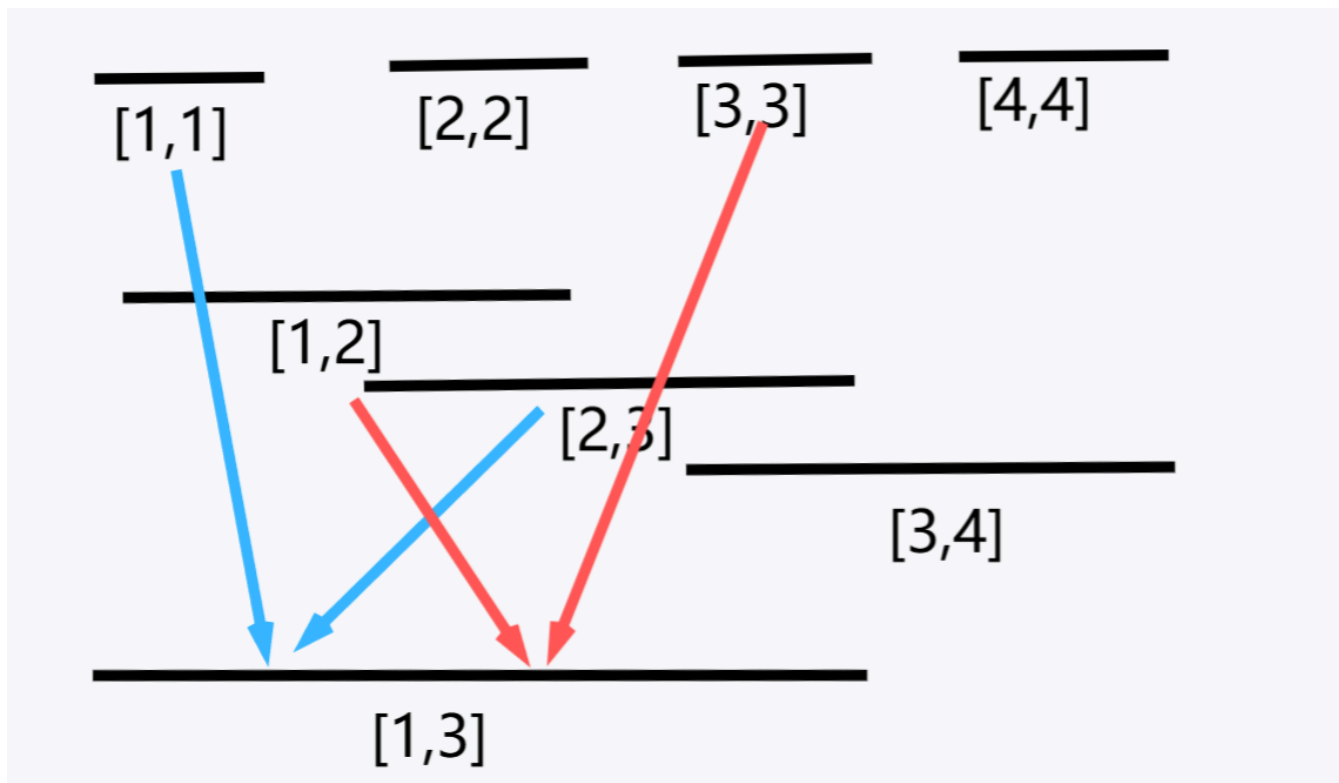
为了更好地演示状态之间的转移，我们先将每个状态都画出来，也就是将这个数组每个可能的连续区间都画出来（按照区间长度的大小、左端点的大小来排序）：



考虑区间长度为二的区间如何转移得到：



可以发现区间长度为二的区间的转移是十分固定的，但是对于区间长度为三及以上的区间的转移就并不固定了，为了演示简洁，我们这里只看区间 $[1, 3]$ 的状态可以由哪些状态转移得到：



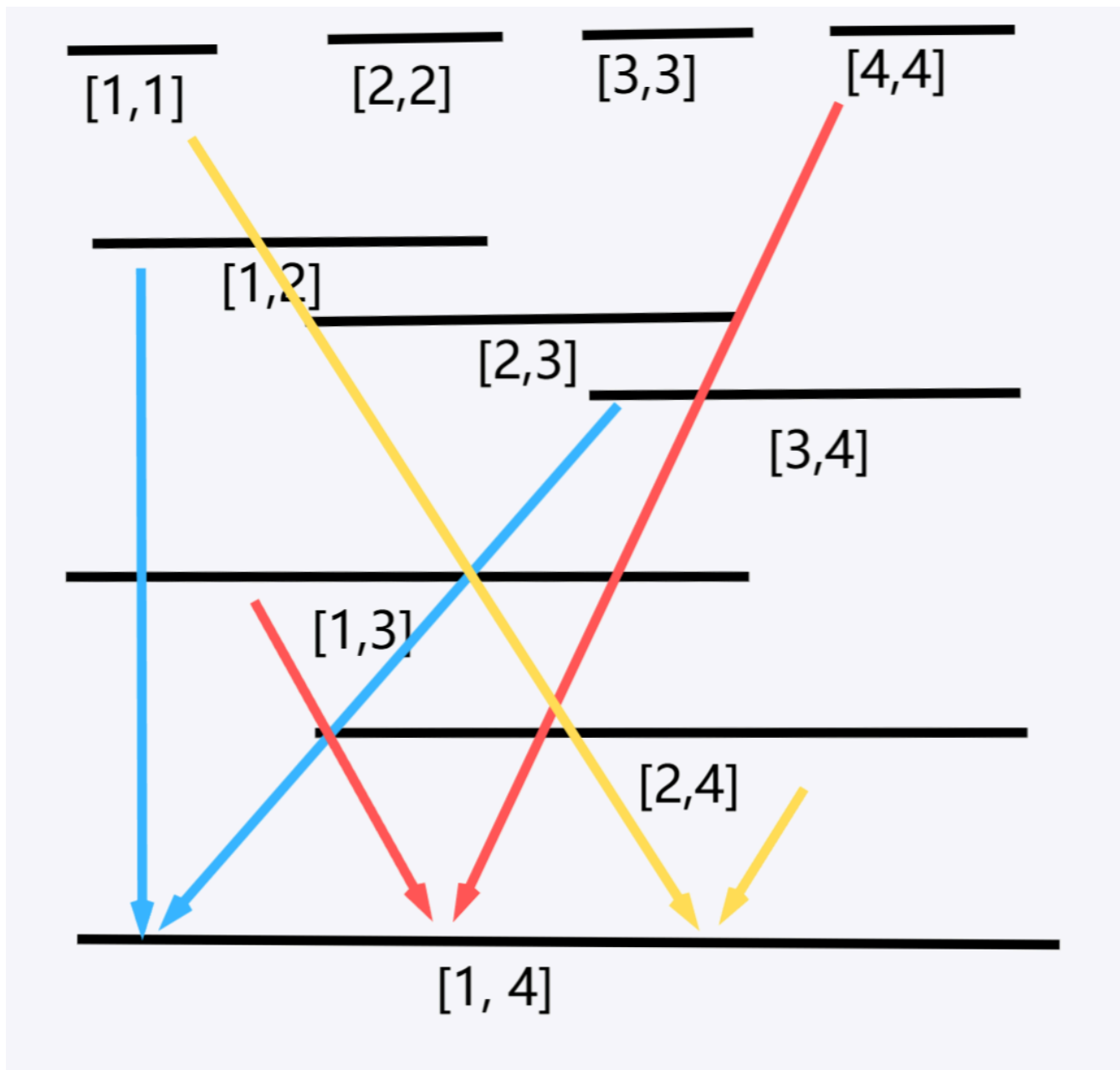
在这里我们可以发现要想转移到 $[1, 3]$ 区间的状态我们有两种不同的方案：可以从 $[1, 1]$, $[2, 3]$ 两个状态转移得到，也可以从 $[1, 2]$, $[3, 3]$ 两个状态转移得到。

但是不会从 $[1, 1]$, $[2, 2]$, $[3, 3]$ 这三个状态转移得到，这是因为我们使用了一个长度为 2 的区间来表示了两个长度为 1 的区间，不然就和暴力没区别了。

事实上，区间 DP 的核心是将问题抽象成一个个子区间的问题，然后考虑可以组成一个大区间的任意两个子区间进行转移的最小代价。

而对于那种转移更好，我们可以通过比较两种方案哪个结果更优来转移。

那么对于长度为 4 的区间状态可以由哪些状态转移得到呢：



可以发现有多种转移方案，而每种方案都是两个连续的子区间来组合出对应的大区间，所以我们可以通过枚举两个子区间的间断点来实现枚举两个子区间。

先解释下为什么区间 DP 的时间复杂度往往是 $O(n^3)$ 的：

令 len 是区间长度， l, r 是区间的左右端点， mid 是将区间分成两个子区间的间断点。

由于我们考虑的状态的顺序是按照区间长度从小到大、左端点从小到大。

我们首先会先用第一层循环来枚举我们考虑当前区间的长度 len , ($len \in [1, n]$)，接下来会枚举当前区间的左端点 l , ($l \in [1, n - len + 1]$)，通过 $r \leftarrow l + len - 1$ 来直接计算出区间的右端点，最后枚举区间的间断点 mid , ($mid \in [l, r - 1]$)，来将区间分为两个子区间 $[l, mid]$, $[mid + 1, r]$ ，这样我们通过三层循环将当前考虑区间和所有可以组合出这个区间的两个连续子区间都枚举出来了。由于是三层循环，所以时间复杂度来到了 $O(n^3)$ 。

但是有些区间 DP 的题目并非一定要 $O(n^3)$ 的复杂度，例如如果一个区间 $[l, r]$ 仅仅只能由 $[l, r - 1]$, $[l + 1, r]$ 两个区间组合得到，那么第三层循环就可以优化掉，时间复杂度仅为 $O(n^2)$ 。

分析

[P1775 石子合并 \(弱化版\)](#)

首先可以注意到 n 是十分小的，允许 $O(n^3)$ 的时间复杂度。其次题目的操作和区间之间的合并有关，所以联想到区间 DP。

定义 $dp_{l,r}$ 为将区间 $[l, r]$ 堆石子合并成一堆的最小代价。由于题目信息已经没有还没表示出来的限制，所以 DP 定义就到此为止了。

接下来考虑初始化，初始化一般要考虑长度为一或者一到二的区间的初始化。但是在这题中，长度为二及以上的区间的合并转移都是一样的，所以仅考虑长度为一的区间合并的值就好，由于长度为一本身就是一堆了，所以不需要代价，即 $dp_{i,i} = 0$ 。对于其他长度的区间，依旧是题目问最小代价那么初始化成无穷大即可。

最后考虑转移，正如在概述说的那样，仅考虑是不是两个连续区间合并成一个大区间（如果要三个或多个区间合并那么需要重新设计 DP 定义）还有合并的代价是多少。可以发现合并成一个大区间 $[l, r]$ 仅需要将两个连续区间 $[l, mid]$, $[mid + 1, r]$ 合并起来就好，并且代价是 $\sum_{i=l}^r m_i$ 。

其中 $\sum_{i=l}^r m_i$ 是将 $[l, mid]$, $[mid + 1, r]$ 这两堆石子合并成一堆的代价，所以还要考虑将 $[l, mid]$, $[mid + 1, r]$ 分别从初始状态合并成一堆的代价，由于我们的 DP 是依照区间长度进行的，而 $[l, mid]$, $[mid + 1, r]$ 两个区间长度一定小于区间 $[l, r]$ 的长度，将 $[l, mid]$, $[mid + 1, r]$ 这两堆石子合并成一堆的代价我们在之前的 DP 转移已经算出来了，依照定义分别是 $dp_{l,mid}$, $dp_{mid+1,r}$ 。而 mid 的取值要在区间 $[l, r - 1]$ 中（枚举是哪两个连续区间合并成 $[l, r]$ 区间）。

所以转移方程为 $dp_{l,r} \leftarrow \min_{mid=l}^{r-1} (dp_{l,mid} + dp_{mid+1,r}) + \sum_{i=l}^r m_i$ 。

最后考虑答案，我们求得是将区间 $[1, n]$ 堆石子合并成一堆的最小代价，依照定义可知代价为 $dp_{1,n}$ 。

DP 设计

定义： $dp_{l,r}$ 表示将区间 $[l, r]$ 堆的石子合并成一堆的最小代价。

初始化： $dp_{i,i} = 0$ 。

转移： $dp_{l,r} \leftarrow \min_{mid=l}^{r-1} (dp_{l,mid} + dp_{mid+1,r}) + \sum_{i=l}^r m_i$ 。

代码

```
1  const long long LINF = 1e18;
2
3  int n;
4  long long a[N], dp[1000][1000], sum[1000];
5  // dp[l][r] 表示区间 [l, r] 堆的石子合并成一堆的最小代价。
6  // sum[i] 表示区间 [1, i] 的石子的质量和（前缀和数组）
7
8  signed main() {
9      cin >> n;
10     for (int i = 1; i <= n; i++) cin >> a[i];
11     for (int i = 1; i <= n; i++) sum[i] = sum[i - 1] + a[i]; // 维护前缀和数组
12
13     // 初始化
14     for (int i = 0; i <= n; i++)
15         for (int j = 0; j <= n; j++)
16             dp[i][j] = LINF;
17 }
```

```

18     for (int len = 1; len <= n; len++) {
19         for (int l = 1; l + len - 1 <= n; l++) {
20             int r = l + len - 1;
21             // 枚举区间的长度和左端点，通过计算得到区间的右端点来实现依据区间的长度，从小到大枚举区间
22
23             if (l == r) {
24                 dp[l][r] = 0; // 初始化
25                 continue;
26             }
27
28             for (int mid = l; mid < r; mid++) {
29                 dp[l][r] = min(dp[l][r], dp[l][mid] + dp[mid + 1][r] + sum[r] - sum[l -
30 1]);
31                 // dp 的转移，通过枚举区间的分阶段来将区间分割成两个连续的区间，判断是否由这两个区间的
32                 // 合并来转移到当前区间
33             }
34         }
35     }
36
37     cout << dp[1][n] << '\n'; // 答案便是将 [1, n] 区间所有石子合并的代价
38
39     return 0;
40 }

```

树形 DP

概述

由于树形 DP 涉及到了如何去遍历一棵树，我会先讲述一些前置知识，然后进行树形 DP 的讲解。

首先先来介绍下我比较常用的代码：

```

1 | vector<int> go[N];

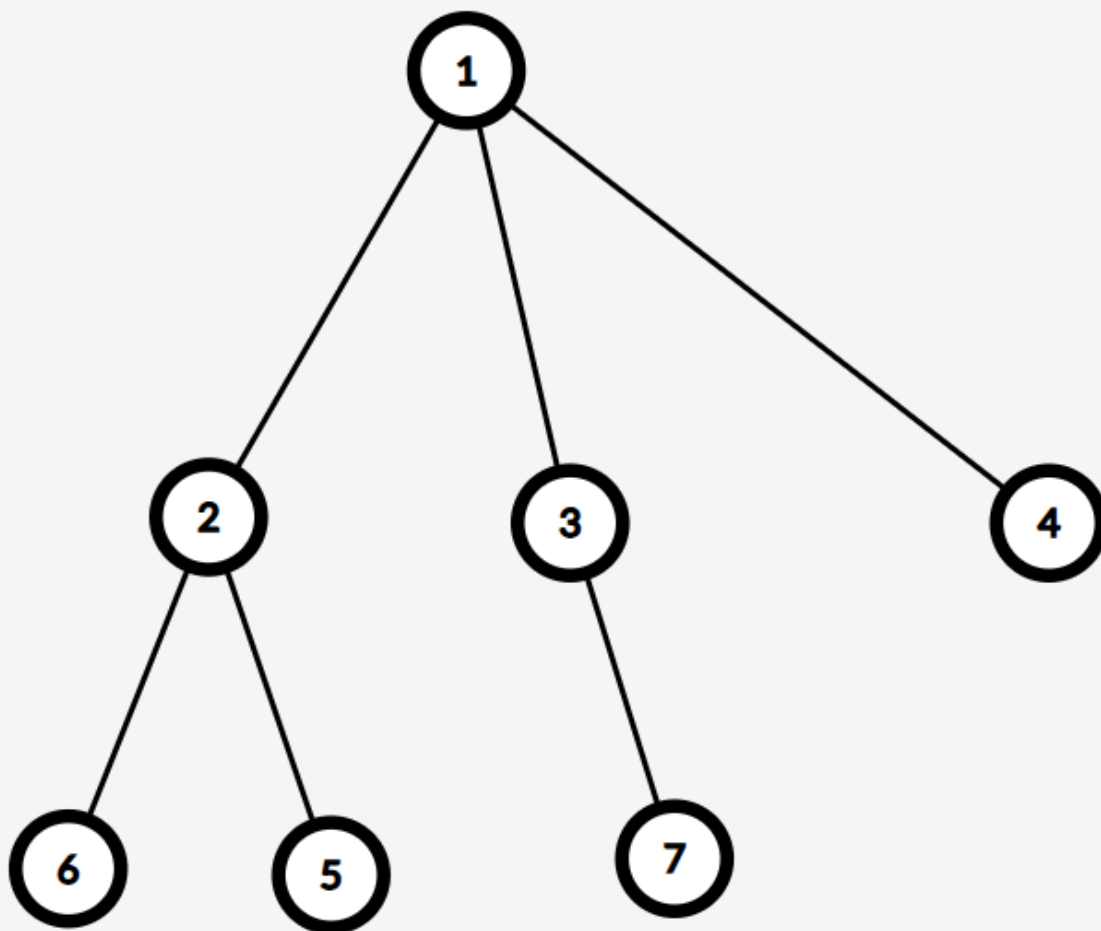
```

其中 `vector<int>` 表示开一个**不定长的 int 类型的数组**，所以，`go` 数组其实是一个**二维数组**。而 go_u 的变量类型是 `vector<int>`，也就是对于一开始的 go_0 到 go_{N-1} ，它们一开始都是一个长度为 0 的**空数组**。可以通过 `go[u].push_back(x)` 函数来将元素 x 放入到数组 go_u 的末尾。

例如一开始 $go_1 = \{\}$ 表示空数组，然后执行 `go[1].push_back(2)`，那么 $go_1 = \{2\}$ 。

go_u 存的是一个数组，这个数组的内容是与 u 连边的所有点的编号。

例如要存储一棵树：



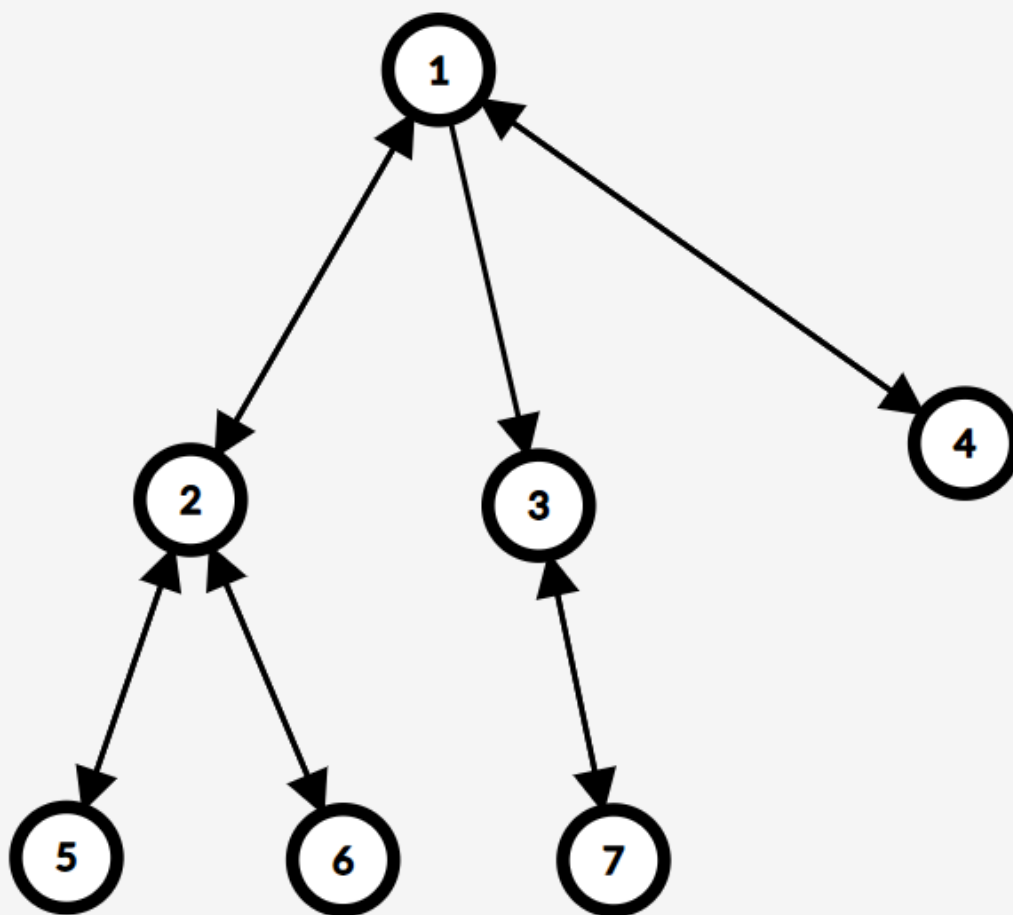
对于点 1 来说，与 1 连边的集合为 2, 3, 4，所以与之对应的 go_1 数组存的内容便是 $go_1 = \{2, 3, 4\}$ 。

对于点 2 来说，与 2 连边的集合为 1, 5, 6，所以与之对应的 go_2 数组存的内容便是 $go_2 = \{1, 5, 6\}$ 。

值得一提的是，存储树的时候我们往往是连无向边，我们使用存边的方式来存树，所以存边的代码为：

```
1  for (int i = 1; i < n; i++) { // i 从 1 到 n - 1 是因为树只有 n - 1 条边
2      int u, v;
3      cin >> u >> v; // 输入表示 u v 连边
4      go[u].push_back(v); // 将 v 存入 go[u]
5      go[v].push_back(u); // 将 u 存入 go[v]
6      // 树一般都是用无向边存储
7  }
```

通过连接无向边，我们用 go 数组得到了以下的树：



存边是为了方便的遍历树，我通常使用 DFS 的方法来进行树的遍历，而 DFS 就需要起点，我们一般选树的根节点来当作 DFS 的起点。

```
1 void dfs(int u, int fa) {  
2     // u 表示当前遍历的节点的编号，fa 是 u 节点父亲编号  
3     for (auto v : go[u]) {  
4         if (v == fa) continue;  
5         // 避免从父亲遍历到儿子，儿子又遍历到父亲的无限循环  
6         dfs(v, u); // 进入下一层迭代，先处理好 v 子树的信息  
7     }  
8 }
```

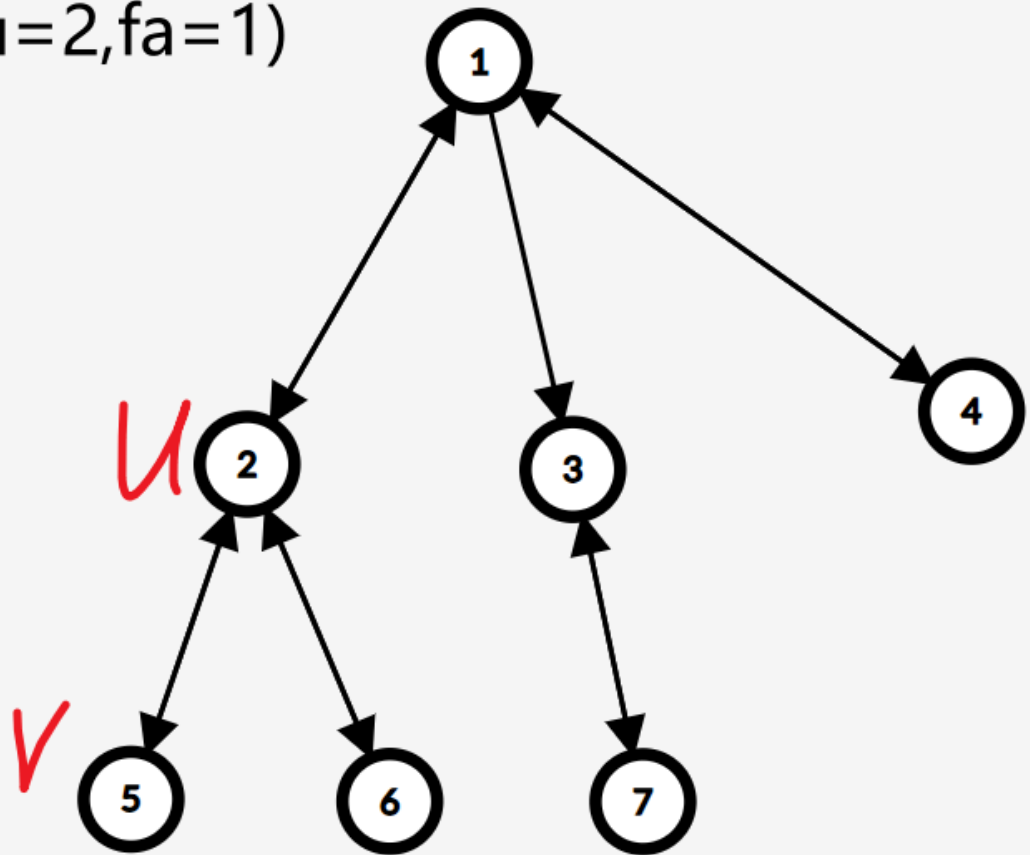
`for (auto v : go[u])` 这行代码是使用 v 来遍历数组 go_u 中的元素，等价于下面的代码：

```
1 for (int i = 0; i < go[u].size(); i++) {  
2     int v = go[u][i];  
3 }
```

例如图中的 go_1 ， v 会依次变成 2, 3, 4。

`if (v == fa) continue;` 这行代码是十分关键，例如当前我们在树中的节点 2，当前 $v = 5$ 。

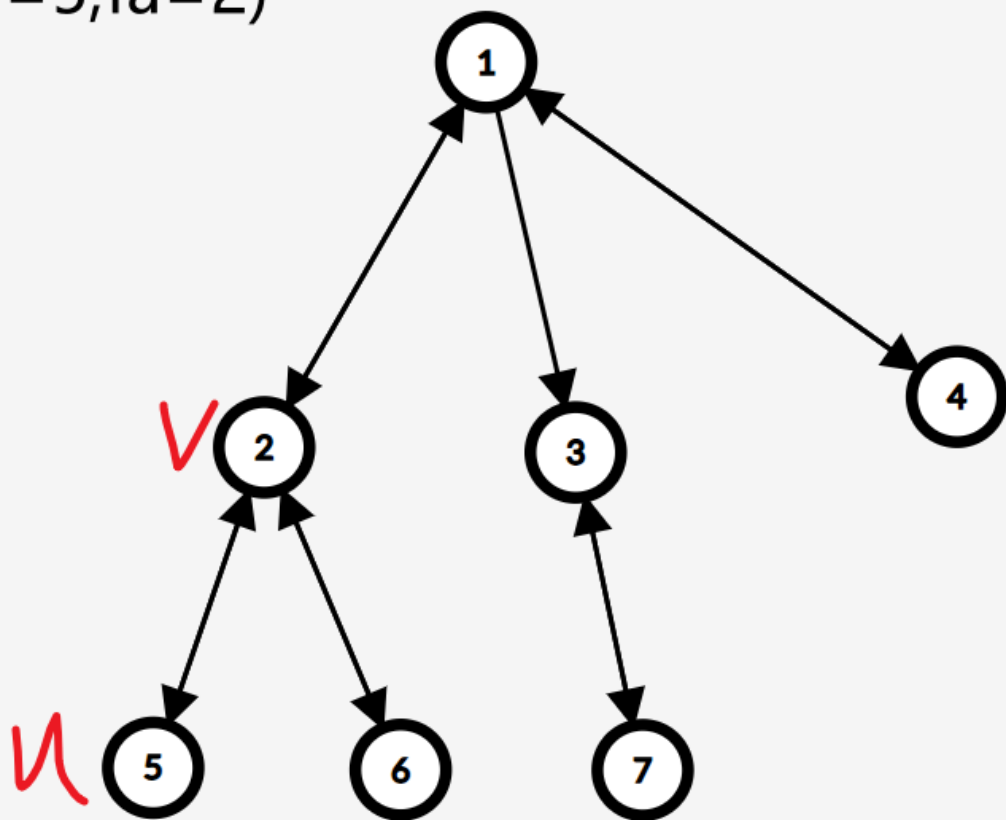
dfs(u=2,fa=1)



接下来会执行 DFS，在刚进入的 DFS 中，我们令 $u' = 5, fa' = 2$ 。

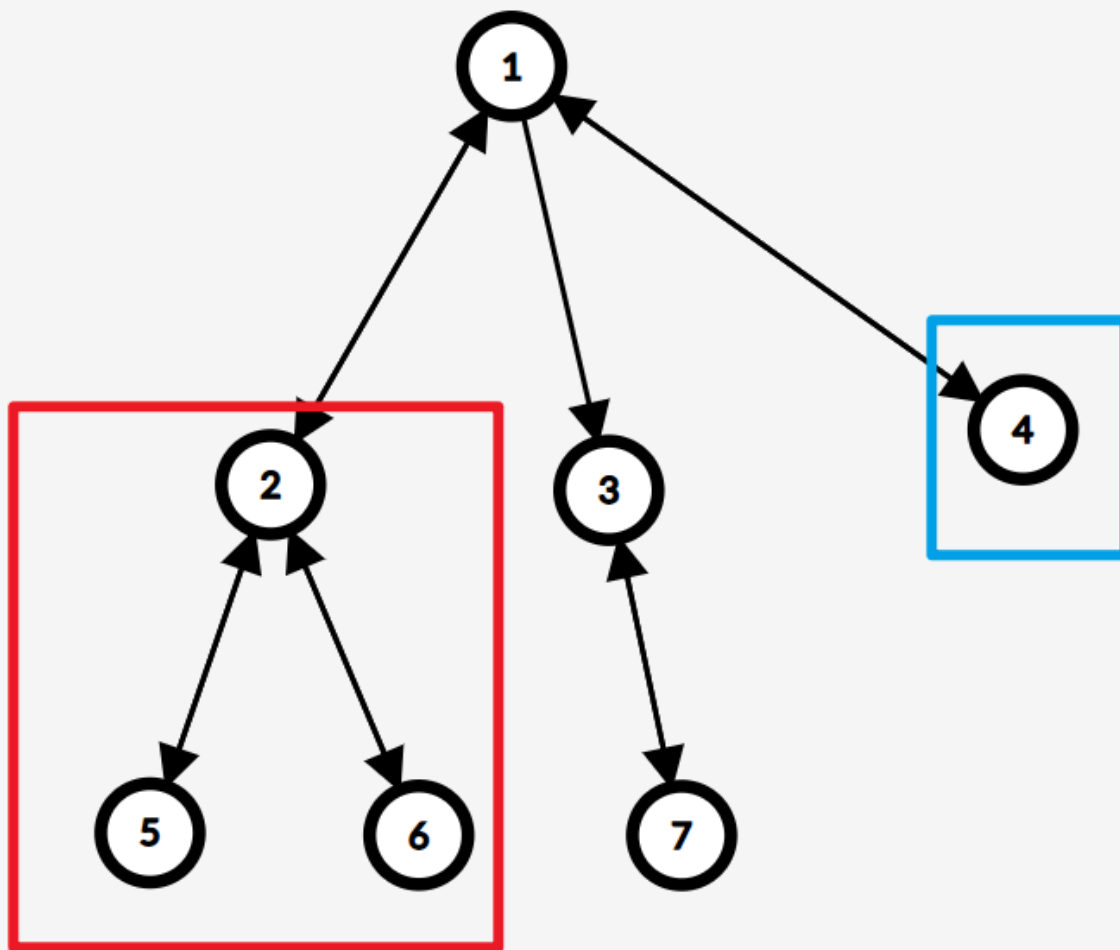
由于 $u' = 5$ 仅和点 2 连边，所以在执行 `for (auto v : go[u])` 时 $v = 2$ 。

dfs(u=5,fa=2)



如果没有 `if (v == fa) continue;` 的话，那么我们会再次执行 DFS 进入 $u'' = 2, fa'' = 5$ 的 DFS，那么程序就会不断循环 2 走向 5，5 又走向 2 的无限循环。

对于**子树**，可以理解为**有根树**中，以一个点及其儿子和孙子组成的树的集合，例如上图中的树，以 2 为根的子树（红框）和以 4 为根的子树（蓝框）分别为：



整体的代码如下：

```
1  int n; // 这棵树有 n 个节点
2  vector<int> go[N]; // go[u] 存的是一个数组，这个数组的内容是与 u 连边的所有点的编号
3
4  void dfs(int u, int fa) { // u 表示当前遍历的节点的编号，fa 是 u 节点父亲编号
5
6      for (auto v : go[u]) {
7          if (v == fa) continue; // 避免从父亲遍历到儿子，儿子又遍历到父亲的无限循环
8          dfs(v, u); // 这一步相当于先把 v 节点的数组信息都算出来
9      }
10 }
11
12 int main() {
13     cin >> n;
14     for (int i = 1; i < n; i++) {
15         int u, v;
16         cin >> u >> v; // 输入表示 u v 连边
17         go[u].push_back(v); // 将 v 存入 go[u]
18         go[v].push_back(u); // 将 u 存入 go[v]
19         // 树一般都是用无向边存储
20     }
```

```
21     dfs(1, 1); // 如果根节点不是1, 把这两个参数都换成根节点
22 }
```

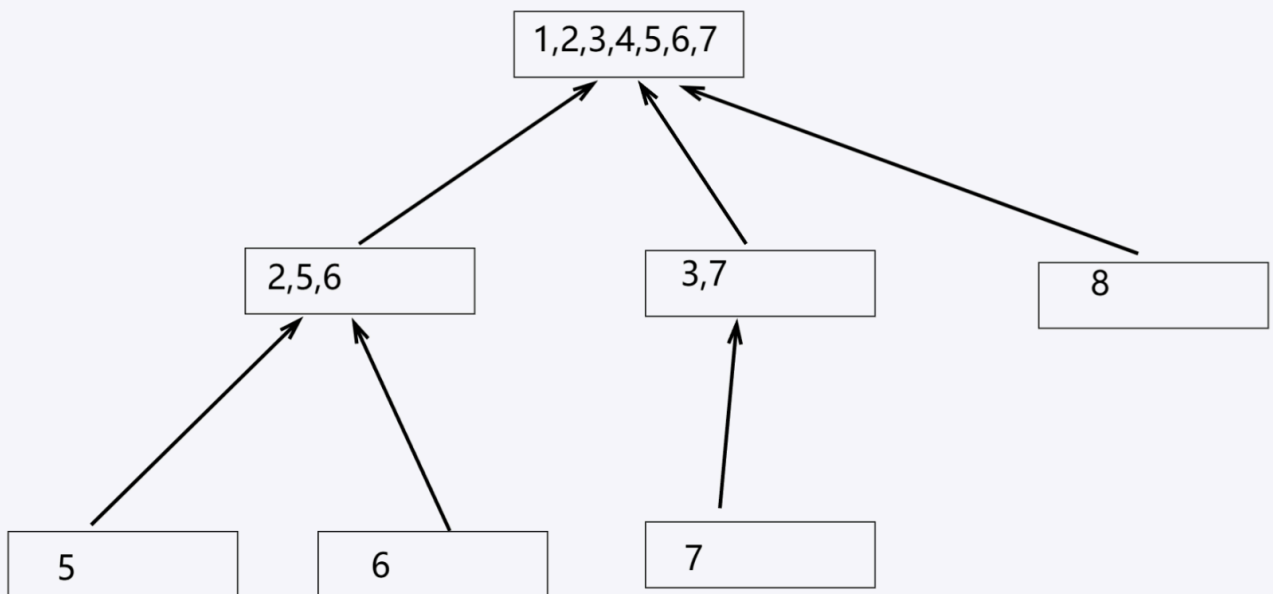
以下是处理一些常见的树上信息的代码：

```
1  int n; // 这棵树有 n 个节点
2  vector<int> go[N]; // go[u] 存的是一个数组, 这个数组的内容是与 u 连边的所有点的编号
3  int sz[N], dep[N], f[N], leaf[N];
4  // sz[u]: u 子树的节点数量
5  // dep[u]: u 点在子树的深度 (根节点深度为1)
6  // f[u]: u 节点的父亲节点编号
7  // leaf[u]: u 是不是叶子节点
8
9  void dfs(int u, int fa) { // u 表示当前遍历的节点的编号, fa 是 u 节点父亲编号
10     f[u] = fa;
11     sz[u] = 1;
12     leaf[u] = go[u].size() == 1;
13     dep[u] = dep[fa] + 1;
14
15     for (auto v : go[u]) {
16         if (v == fa) continue; // 避免从父亲遍历到儿子, 儿子又遍历到父亲的无限循环
17         dfs(v, u); // 这一步相当于先把 v 节点的数组信息都算出来
18         sz[u] += sz[v];
19     }
20 }
21
22 int main() {
23     cin >> n;
24     for (int i = 1; i < n; i++) {
25         int u, v;
26         cin >> u >> v; // 输入表示 u v 连边
27         go[u].push_back(v); // 将 v 存入 go[u]
28         go[v].push_back(u); // 将 u 存入 go[v]
29         // 树一般都是用无向边存储
30     }
31     dfs(1, 1); // 如果根节点不是1, 把这两个参数都换成根节点
32 }
```

在整理完前置知识后我们来看树形 DP 是如何分析的了。

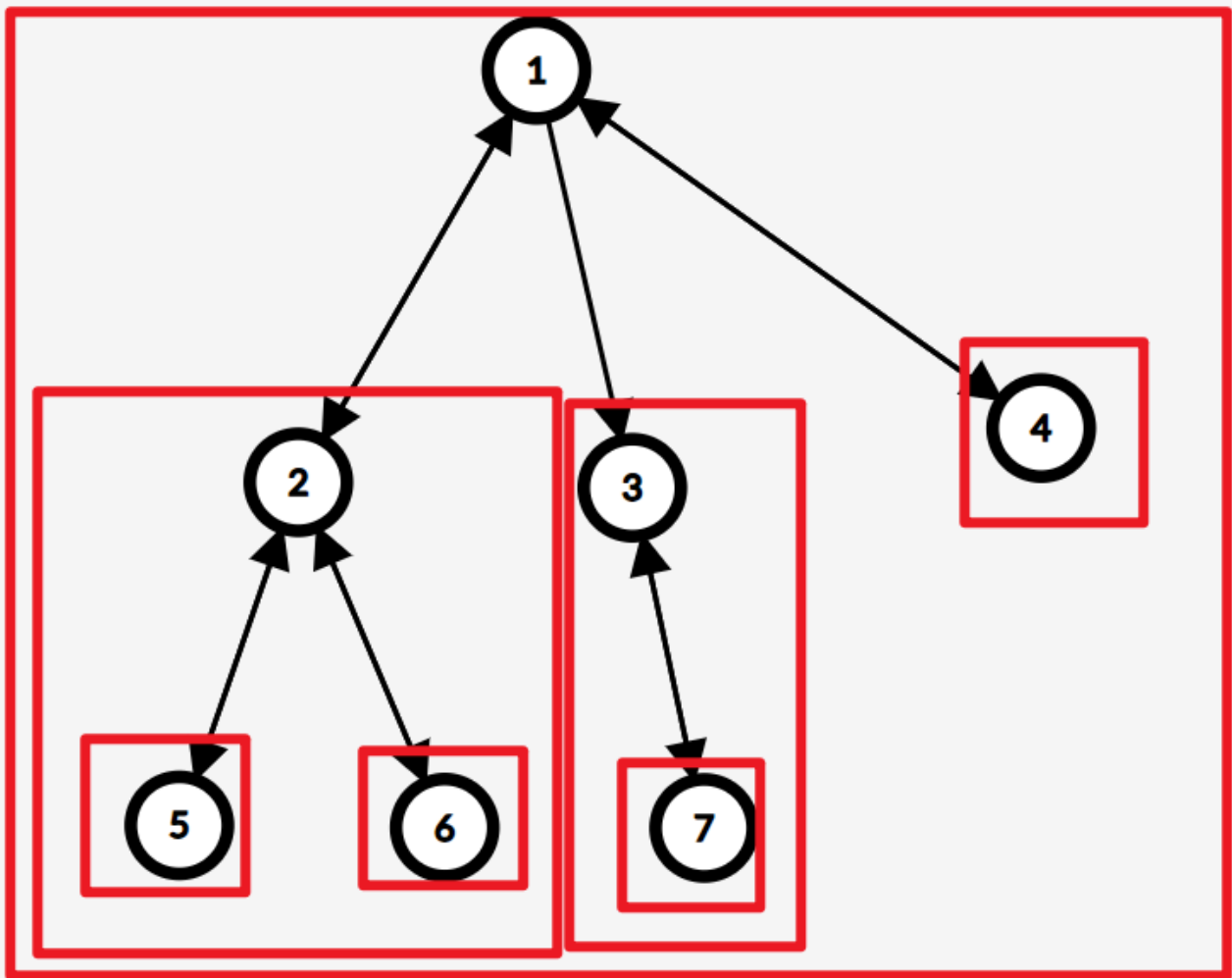
由于我们要保证状态的转移是从小范围状态转移到包含这个小范围的大范围状态, 所以对于树形 DP 我们往往从**子树**的角度出发。

我们可以从状态的角度来看一下状态之间的转移关系：



可以看到状态之间的转移和树的结构完全一致，并且每个状态仅仅由一个节点的直接儿子转移，所以转移关系为 $dp_u \leftarrow f(dp_v), v \in son_u$ ，其中 $f(x)$ 是题目中对应的操作转移代价， son_u 表示 u 的直接儿子的集合。

如果从树的角度看状态的话会是这样：



可以发现每一个状态到另一个状态的转移，都是一个小范围状态转移到包含这个范围的大范围状态。

所以大多数的树形 DP 的定义都是以 dp_u 表示以 u 为根的子树的最优值。

分析

[P1352 没有上司的舞会](#)

首先每个职员只会会有一个直接上司，对应了树型结构中每个点至多有一个父亲，所以我们可以考虑树形 DP。

令 dp_u 表示以 u 为根的子树可以获得的最大快乐值。

但是又发现 v 点是否会对 dp_u 产生 r_v 的贡献这和 u 职员是否参与聚会有关，所以为了知道 u 是否参与聚会，所以我们对 dp_u 再增加一个维度，用于表示 u 不来/来参加聚会。

令 $dp_{u,0/1}$ 表示以 u 为根的子树中， u 职员不来/来参加聚会可以获得的最大快乐值。

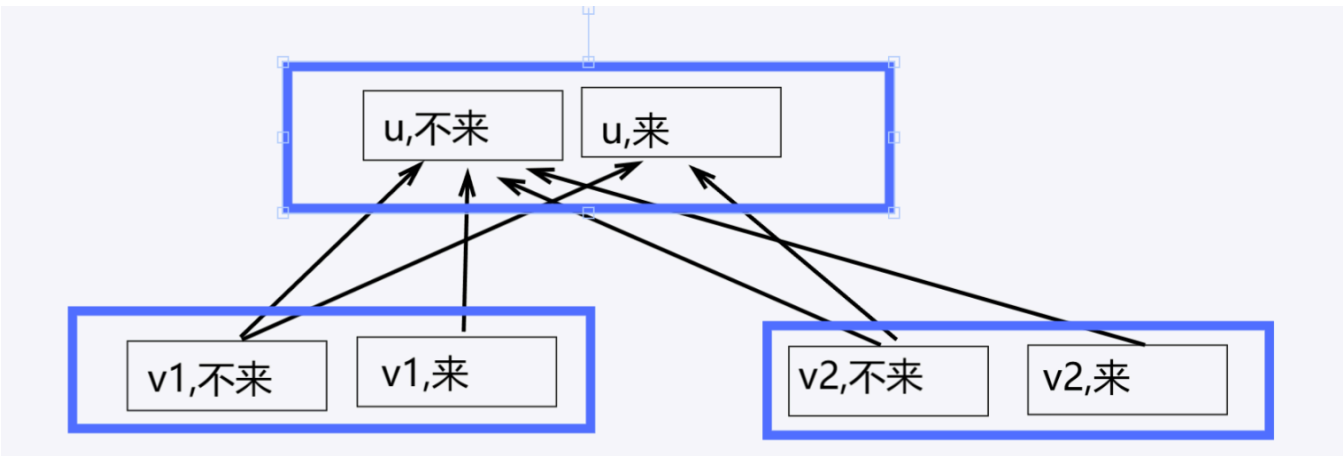
考虑初始化，一开始所有人不参加聚会，那么会有 $dp_{u,0} = 0$ ，如果 u 参加聚会，那么至少会有 r_u 的贡献，那么有 $dp_{u,1} = r_u$ 。

最后考虑转移，由于树形 DP 的转移仅和当前节点和其直系儿子节点有关，所以我们讨论所有情况：

1. u 职员不参加聚会，那么对于其下属 $v_1, v_2 \dots v_k, v_i \in son_u$ 来说，他们可以来可不来，所以取他们不来/来两种情况快乐值最大的情况就好，即 $dp_{u,0} \leftarrow \sum_{v \in son_u} \max(dp_{v,0}, dp_{v,1})$ 。
2. u 职员参加聚会，那么对于其下属 $v_1, v_2 \dots v_k, v_i \in son_u$ 来说，他们是不可以来参加聚会的，所以只能取他们不来这一种情况进行转移，即 $dp_{u,1} \leftarrow \sum_{v \in son_u} dp_{v,0}$ 。

最后考虑答案，我们想要知道全局的最优值，那么 dp_1 便是表示了整棵树也就是全局的答案，分为 1 号职员不来/来两种情况，我们取最大的输出即可，即 $ans \leftarrow \max(dp_{1,0}, dp_{1,1})$ 。

当然也可以从状态的角度理解转移，由于每个点有不来/来两种状态，所以要将树上的节点拆成两个状态，我们以一个比较简单的结构举例子：



每个蓝框表示一个节点，蓝框内可以根据题目的限制拆分为多种状态。

如果 u 不来的话，那么在 v_i 对应的蓝框内选值最大的状态进行转移就好；如果 u 来的话，只能在每个蓝框选择 v_i 不来的状态来转移了。

DP 设计

定义： $dp_{u,0/1}$ 表示以 u 为根的子树中， u 点代表的职员不来/来可以得到的最大快乐指数。

初始化： $dp_{u,0} = 0, dp_{u,1} = r_u$ ，其中 r_u 表示 u 职员的快乐值。

转移： $dp_{u,0} \leftarrow \sum_{v \in son_u} \max(dp_{v,0}, dp_{v,1})$ ， $dp_{u,1} \leftarrow \sum_{v \in son_u} dp_{v,0}$ ，其中 son_u 表示 u 点的直接儿子的集合。

代码

```
1  int n, a[N], dp[N][2];
2  vector<int> go[N]; // go[u] 存的是一个数组，这个数组的内容是与 u 连边的所有点的编号
3
4  void dfs(int u, int fa) { // u 表示当前遍历的节点的编号，fa 是 u 节点父亲编号
5      dp[u][1] = a[u]; // 初始化，员工 u 来的话至少会有其自身的快乐值
6      for (auto v : go[u]) { // 遍历与 u 点连边的点的集合
7          if (v == fa) continue; // 避免从父亲遍历到儿子，儿子又遍历到父亲的无限循环
8          dfs(v, u);
9          dp[u][0] += max(dp[v][0], dp[v][1]); // u 员工不来的话，其下属可以选择不来/来
10         dp[u][1] += dp[v][0]; // u 员工来的话，其下属一定不能来
11     }
12 }
13
14 void solve() {
15     std::cin >> n;
16     for (int i = 1; i <= n; i++) cin >> a[i]; // 输入职工 i 的快乐值
17     for (int i = 1; i < n; i++) {
18         int u, v;
19         cin >> u >> v; // 输入表示 u v 连边
20         go[u].push_back(v); // 将 v 存入 go[u]
21         go[v].push_back(u); // 将 u 存入 go[v]
22     }
23
24     dfs(1, 1); // 如果根节点不是1，把这两个参数都换成根节点
25
26     std::cout << max(dp[1][0], dp[1][1]); // 分为职工 1 不来/来两种情况，取最大值
27 }
```

第五讲 DP的选取方案

概述

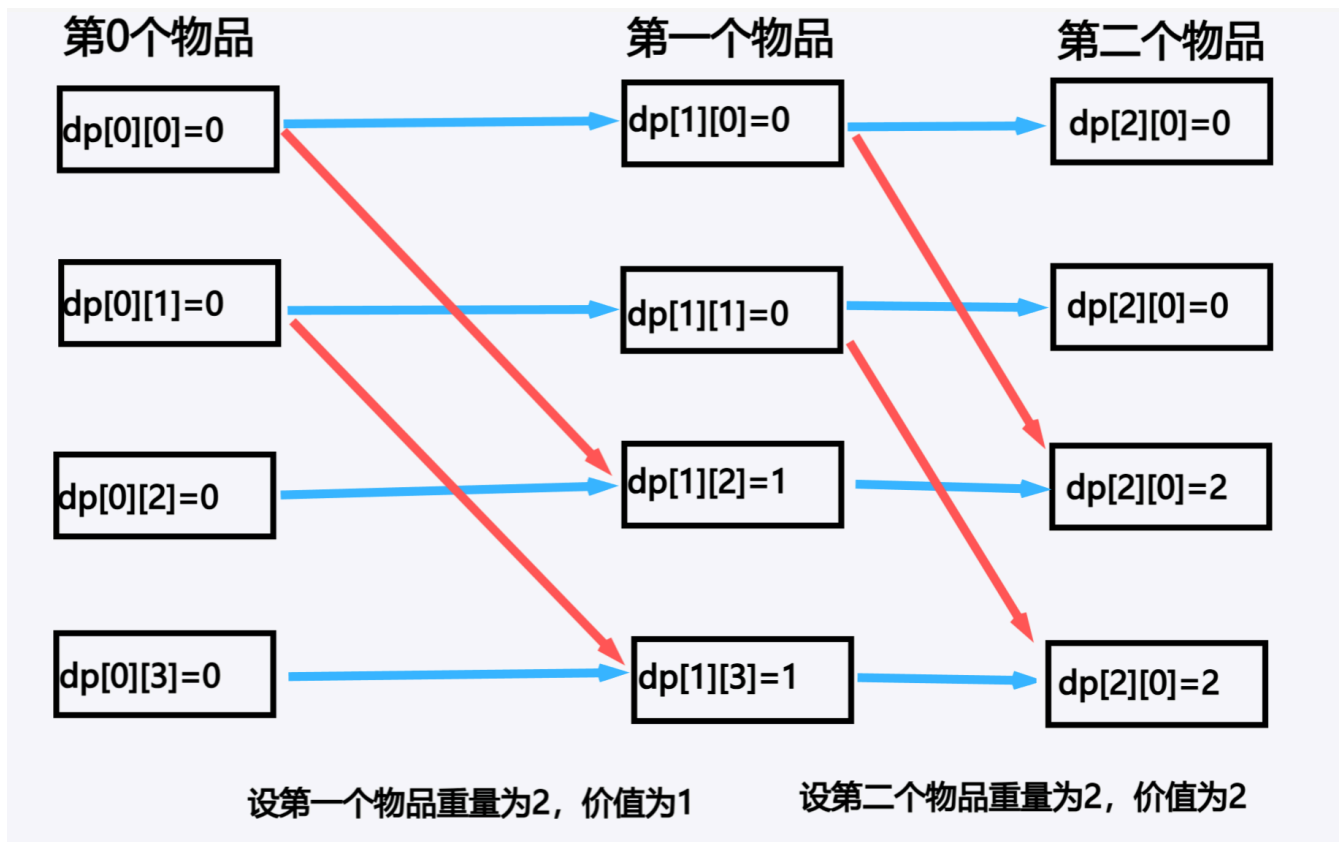
输出 DP 的选取方案的题目一般要分成两个步骤：1. 先用 DP 解决问题的最优解。2. 记录最优解的状态，反向推导到最初状态。

我们从状态的角度理解会比较好，先假设我们已经完成了一道 01 背包问题的 DP：

定义： $dp_{i,j}$ 表示前 i 个物品的选取使用了**不超过** j 的空间可以获得的最大价值。

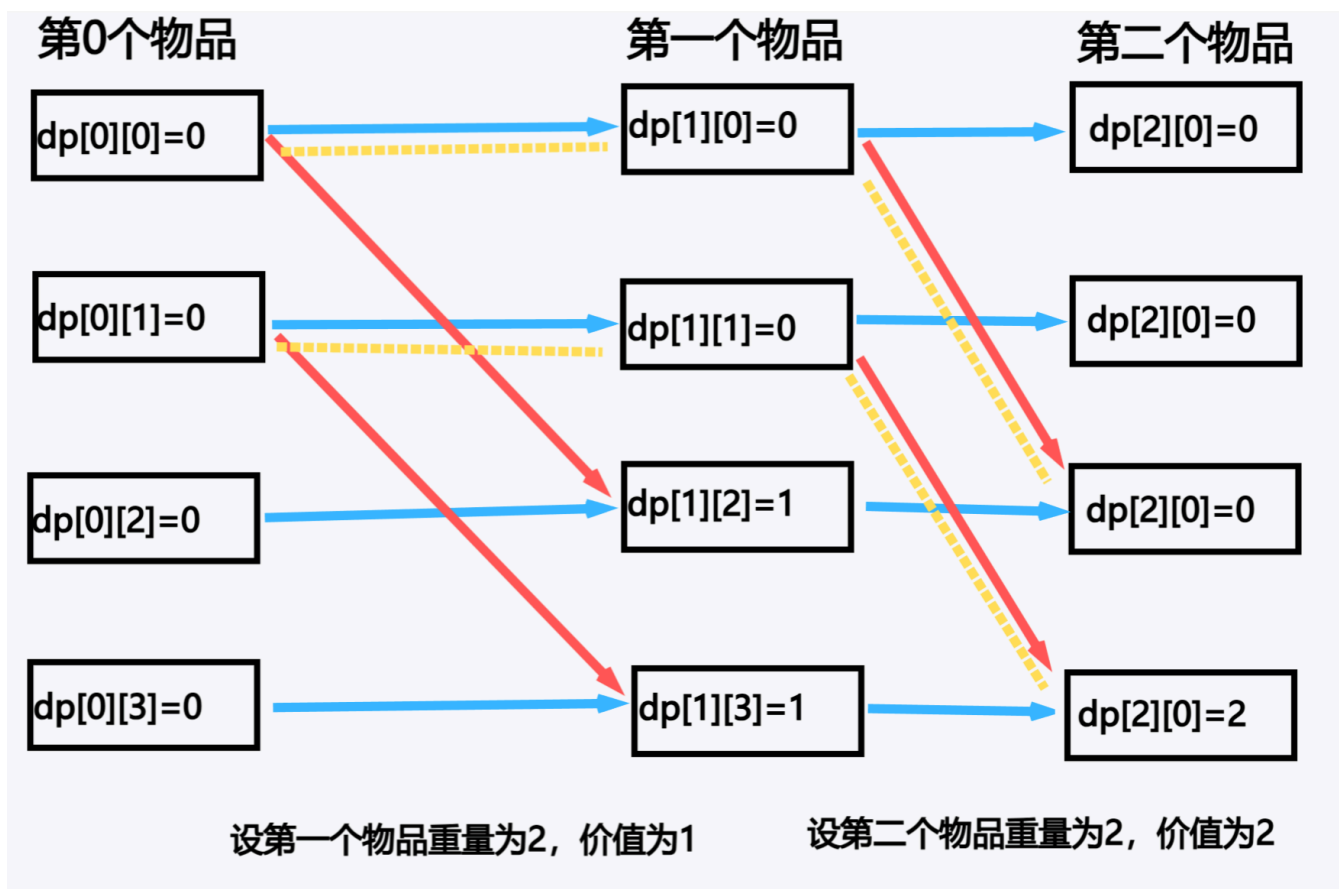
初始化： $dp_{i,j} = -\infty, dp_{0,j} = 0$ 。

转移： $dp_{i,j} \leftarrow \max(dp_{i-1,j}, dp_{i-1,j-weight_i} + val_i)$ 。



其中蓝色箭头表示 $dp_{i,j} \leftarrow dp_{i-1,j}$ 的转移（即不选择第 i 个物品），红色箭头表示 $dp_{i,j} \leftarrow dp_{i-1,j-weight_i} + val_i$ 的转移（即选择第 i 个物品）。

然后将得到最优状态的路径画出来（使用黄色虚线表示路径）：

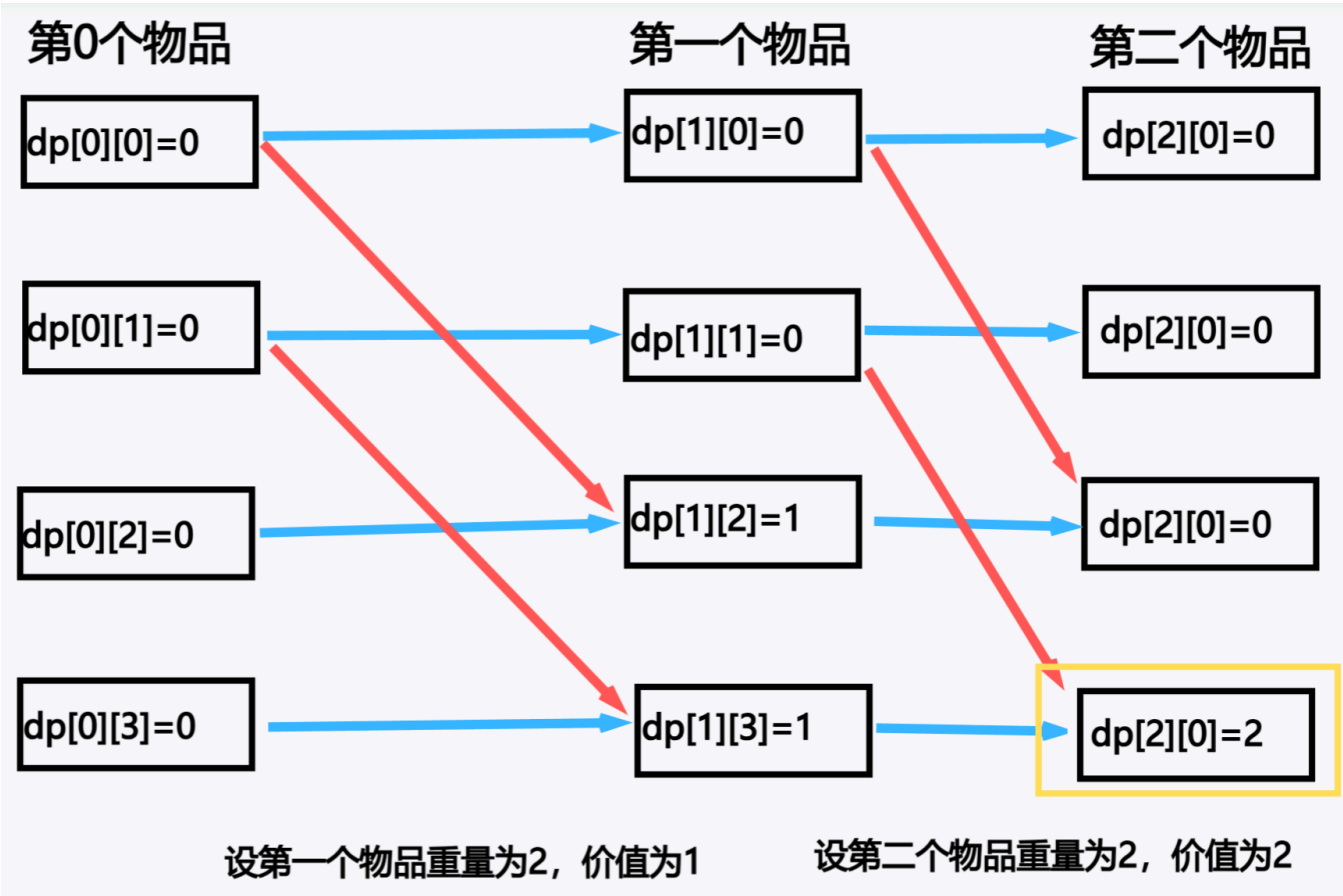


可以看出路径并非只有一条，我们选择其中一条即可。当然如果题目有限制的话，就需要具体分析了（例如下面的例题就要求字典序最小）。

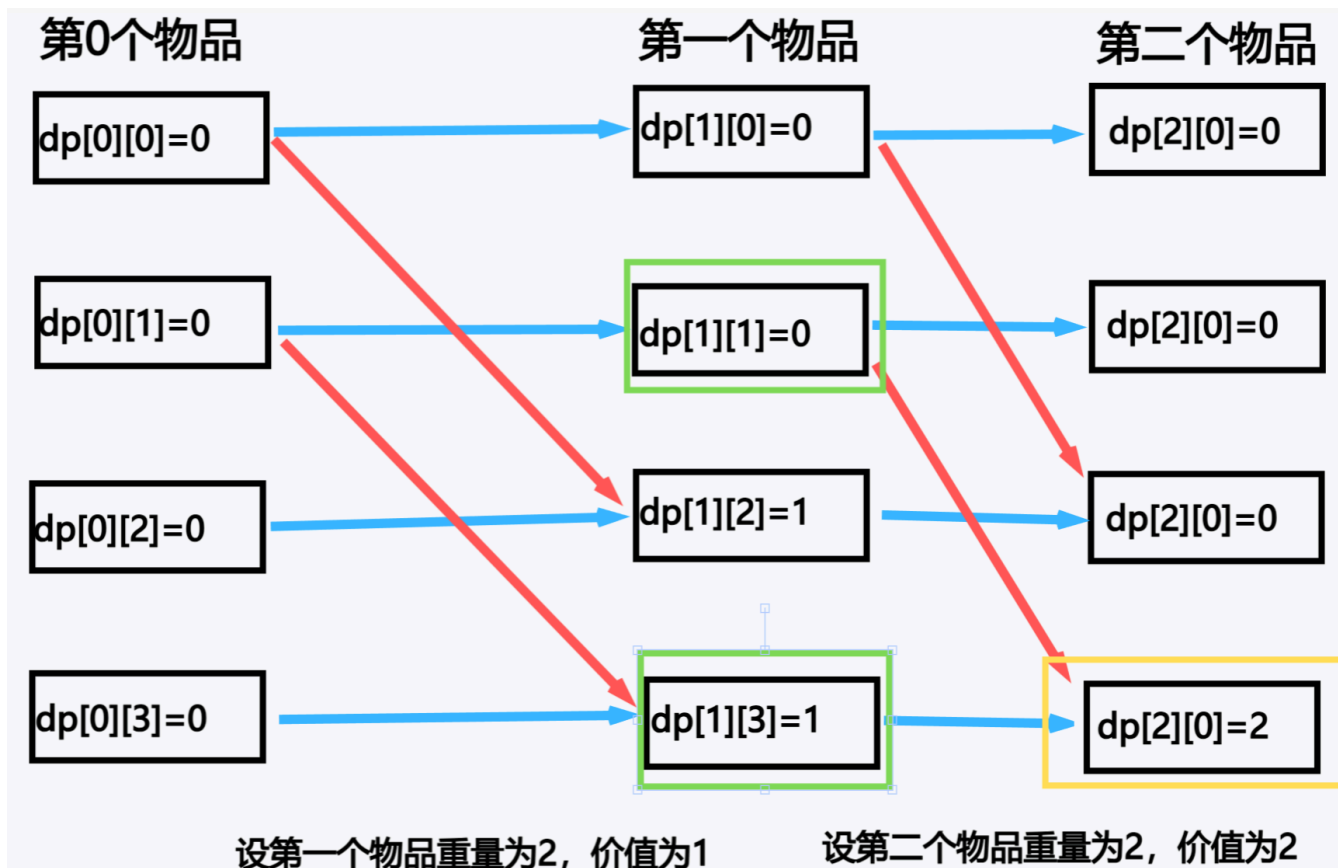
我们进行一次 DP 求解便是我们的第一步，接下来是进行第二步从最优状态反向推导到最初状态。

依据我们的 DP 定义，我们可以知道答案为 $dp_{n,m}$ （表示前 n 个物品使用了不超过 m 的空间获得的最优值）。也就是最优状态在第 n 行 m 列。

以此状态为起点，令 $N = n, M = m$ 来记录我们当前所在的状态（使用黄色方框表示我们当前所用 N, M 记录的状态）：

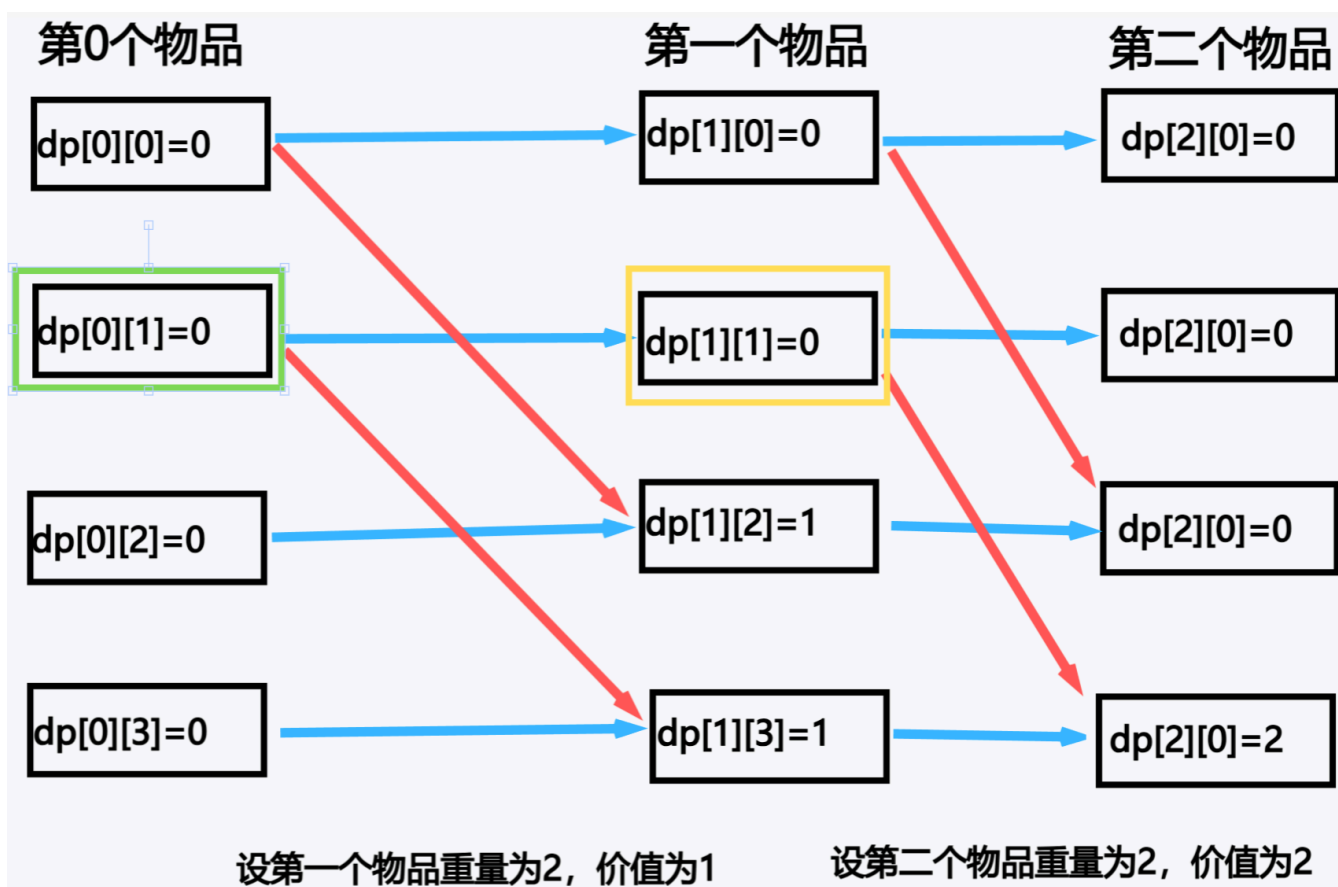


依照指向它的箭头反向考虑是从哪个状态转移来的，可以看到当前状态是由绿色方框括起来的两个状态比较后转移得到：



由于 $dp_{1,1+weight_2} + val_2 > dp_{1,3}$, 可知是从 $dp_{1,1}$ 这个状态转移得到的, 也就是我们选择了第 2 个物品加入到了背包。

既然第 2 个物品已经确定是选择的, 那么就继续考虑第 1 个物品的选取, 首先将 N, M 更新为 $N \leftarrow N - 1, M \leftarrow M - weight_2$, 更新后如下图所示:



由于仅能由 $dp_{0,1}$ 这一个状态转移得到，所以第 1 个物品是不能放进背包的。

而 $dp_{0,1} = 0$ 这个状态是我们的初始化中得到的，所以我们进入了 DP 时的最初状态，也就是说我们已经考虑完了 DP 的选取方案，使用一个容器存储路径最后输出即可。

分析

[P1759 通天之潜水](#)

由于题目中限制了我们输出的方案要保证字典序最小，所以在这里我们的 DP 设计需要具体分析。

定义： $dp_{i,j,k}$ 表示前 i 个物品使用了**不超过** j 重量且**不超过** k 阻力能停留的最久时间。

这里不使用**恰好**来进行定义，是因为在**不超过**的定义中，我们的最优状态一定是题目给出的限制的最大范围，如果使用**恰好**的定义的话，首先我们需要循环来单独确定哪个是最优状态，其次由于会有多个状态的值都是最优的，我们要确定以哪个状态为起点来倒推路径能够得到字典序最小的方案是很麻烦的。

依照我们第三讲的 DP 设计以及第四讲的背包 DP 例题，我们可以发现这题是一道二维背包问题，所以我们仅需在一维背包的 DP 定义的基础上再增加一维用于表示对于阻力的限制即可。

综上所述，我们得到了定义： $dp_{i,j,k}$ 表示前 i 个物品使用了**不超过** j 重量且**不超过** k 阻力能停留的最久时间。

其次是初始化，由于要求字典序最小，所以在我们贪心的选取方案时，应该保证尽可能选取数字小的方案，所以我们选方案时要从第 1 个物品是否选择考虑到第 n 个物品是否选择。由于选方案是 DP 过程的倒序，所以我们在 DP 转移时就应该从 n 到 1 的转移，同样的，初始化也就不能去初始化 dp_0 的状态，而应该初始化 dp_{n+1} 的状态。

所以我们的初始化为： $dp_{i,j,k} = -INF, dp_{n+1,j,k} = 0$ 。

最后看转移，就像刚刚说的，选方案要从 1 到 n 考虑，DP 的过程只能和选方案反着来，所以要倒序转移，那么对于第 i 个物品的选择应该从第 $i + 1$ 个物品的选择的基础上考虑。

所以转移为： $dp_{i,j,k} \leftarrow \max(dp_{i+1,j,k}, dp_{i-1,j-a_i,k-b_i} + c_i)$ ，其中 a_i, b_i, c_i 分别表示第 i 个物品的重量、阻力、可供停留时间。

DP 设计

定义： $dp_{i,j,k}$ 表示前 i 个物品使用了不超过 j 重量且不超过 k 阻力能停留的最久时间。

初始化： $dp_{i,j,k} = -INF, dp_{n+1,j,k} = 0$ 。

转移： $dp_{i,j,k} \leftarrow \max(dp_{i+1,j,k}, dp_{i-1,j-a_i,k-b_i} + c_i)$ ，其中 a_i, b_i, c_i 分别表示第 i 个物品的重量、阻力、可供停留时间。

代码

```
1  const int INF = 0x3f3f3f3f;
2  int m, v, n;
3  int a[101], b[101], c[101];
4  int dp[102][201][201]; // dp[i][j][k] 表示前 i 个物品使用了不超过 j 重量且不超过 k 阻力能停留的
    最久时间
5  int path[101], id; // 用于存储方案
6
7  signed main() {
8      cin >> m >> v >> n;
9      for (int i = 1; i <= n; i++) cin >> a[i] >> b[i] >> c[i];
10 }
```

```

11 // 初始化
12 for (int i = 0; i <= n; i++)
13     for (int j = 0; j <= m; j++)
14         for (int k = 0; k <= v; k++)
15             dp[i][j][k] = -INF;
16 for (int j = 0; j <= m; j++)
17     for (int k = 0; k <= v; k++)
18         dp[n + 1][j][k] = 0;
19
20 // 转移
21 for (int i = n; i >= 1; i--) {
22     for (int j = 0; j <= m; j++) {
23         for (int k = 0; k <= v; k++) {
24             dp[i][j][k] = dp[i + 1][j][k];
25             if (j < a[i] || k < b[i]) continue;
26             dp[i][j][k] = std::max(dp[i + 1][j][k], dp[i + 1][j - a[i]][k - b[i]] +
c[i]);
27         }
28     }
29 }
30
31 cout << dp[1][m][v] << "\n";
32
33 // 考虑 dp 的方案选取
34 int M = m, V = v; // 使用 M, V 来记录我们当前的状态，用 m, v 来确定我们最初的状态（也就是 DP
中的最优状态）
35 for (int i = 1; i <= n; i++) { // DP 的倒序，DP 是 n 到 1，选取就是从 1 到 n。这样的目
的是保证贪心选取时保证字典序最小
36     if (M >= a[i] && V >= b[i] && dp[i][M][V] == dp[i + 1][M - a[i]][V - b[i]] +
c[i]) {
37         // 判断是否选择第 i 个物品，也就是当前状态是否从选择第 i 个物品前的状态转移的来
38         path[++ id] = i; // 满足的话就将选择加入到路径中
39         M -= a[i];
40         V -= b[i];
41     }
42 }
43
44 for (int i = 1; i <= id; i++) cout << path[i] << " "; // 输出路径
45
46 return 0;
47 }

```

后言

我一直认为算法是非常自由的，务必不要把算法看成是一个很独立的东西，所以在这篇文章中我一直都在强调对于 DP 问题的分析。

例如前缀和，我们依旧可以用三个子问题的形式来解决：

1. 定义： sum_i 表示区间 $[1, i]$ 的元素和。
2. 初始化： $sum_1 = a_i$ 。

3. 转移: $sum_i \leftarrow sum_{i-1} + a_i$ 。

事实上，无论是区间 DP 还是树形 DP 亦或是其他，当我们根据题目分析确定 DP 的定义时，其初始化和转移也会跟着被确定，那么这个 DP 的设计也随之被解决了。而我们确定 DP 定义的核心在于对于题目的具体问题具体分析，观察题目的数据范围、操作的特性等来分析出什么定义适合这道题目，切记不要固化自己的思维，比如看到区间 DP 就以为一定是 n^3 时间复杂度。

其次动态规划其实是相当吃熟练度的，很多时候凭直觉就能想到这道题是不是 DP 甚至直接想到适用于这道题的 DP 模型，所以仅仅看这一篇文章来学习是远远不足的，必须配套着大量的刷题训练才能有效果。

最后，祝愿每一位选手都可以在竞赛中享受一份纯粹的快乐。